



Modelos fundacionales en visión artificial

Sergio Villanueva

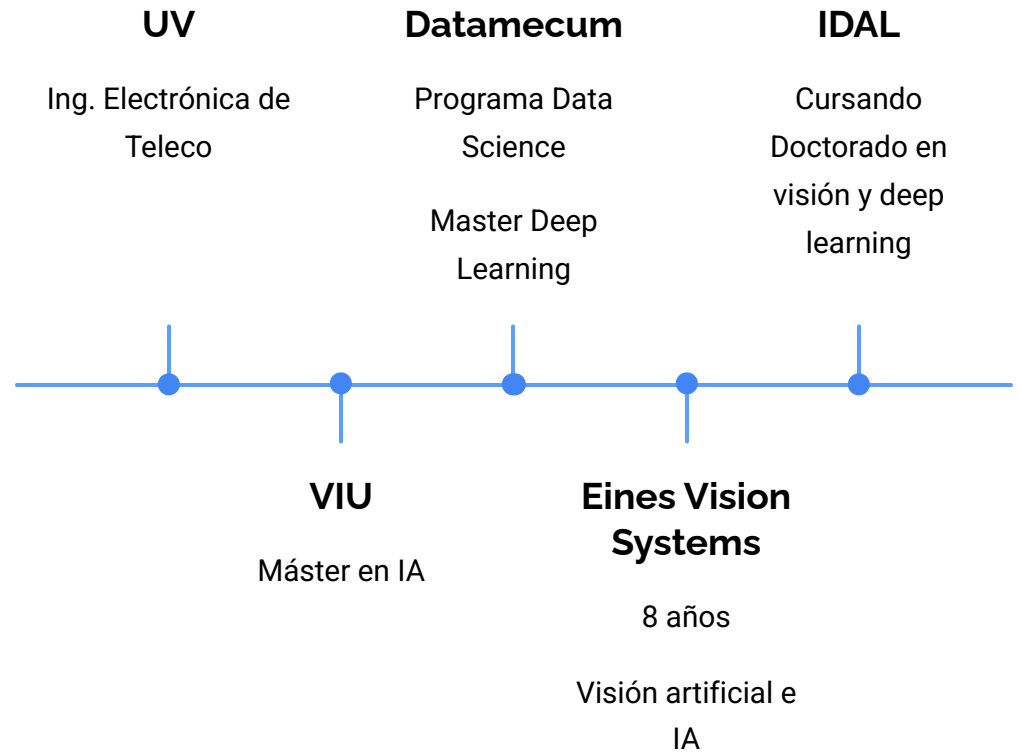
Máster en Deep Learning e IA Generativa
Datamecum

28 y 29 de Noviembre de 2025



Sergio Villanueva

Investigador en Visión Artificial



Temario

Viernes: Fundamentos y ecosistemas

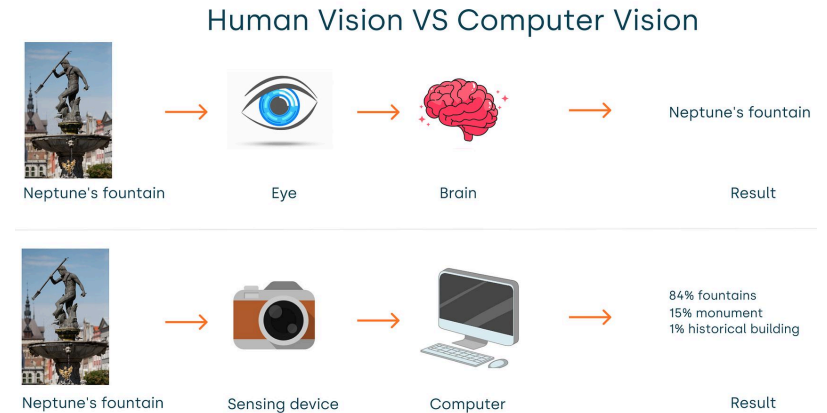
- Detección de Objetos
- Hugging Face
- Modelos Fundacionales Multimodales

Sábado: Avances y aplicaciones

- DINO
- SAM
- Otras Tareas Fundacionales de Visión

Enseñando a las máquinas a ver

La Visión Artificial es la disciplina de la inteligencia artificial que busca capacitar a las máquinas para **extraer información significativa y comprender el mundo a partir de imágenes y vídeos digitales**, imitando la percepción humana.



La brecha semántica: ¿Cómo lo ve un humano vs. una máquina?

Visión humana:

Inmediatamente identificamos "un gato naranja durmiendo en un sofá" y comprendemos su contexto.

Visión computacional:

Una máquina percibe una compleja **matriz de millones de píxeles**, cada uno con valores numéricos RGB como [255, 128, 64...].

Tareas principales en visión artificial

Clasificación

Identifica qué objetos están presentes en una imagen.

Detección de objetos

Localiza y clasifica los objetos individuales dentro de un recuadro delimitador.

Segmentación

Define el contorno exacto de cada objeto a nivel de píxel.

Detección de anomalías

Identifica eventos o datos inusuales que no coinciden con un patrón esperado.

Estimación de pose

Localiza puntos clave (articulaciones) en personas o animales para comprender su postura.

Otras

OCR, Estimación de profundidad, Super resolución, keypoint matching...

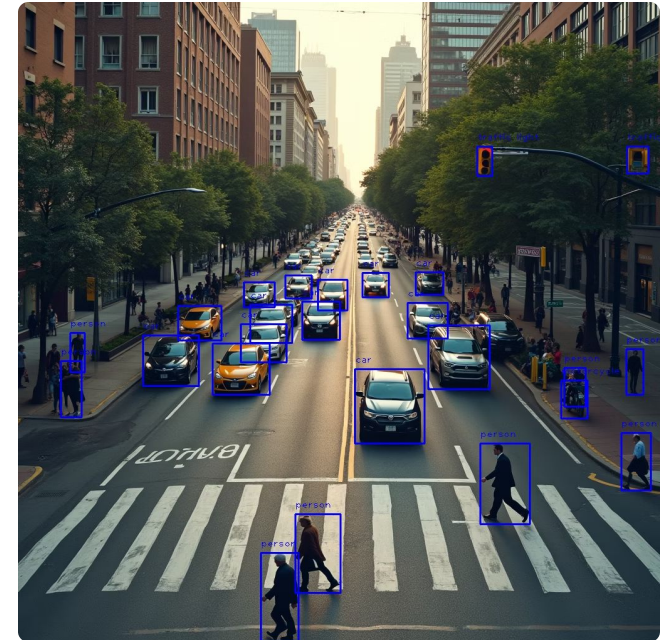


1. Detección de objetos

La detección de objetos va más allá de la simple clasificación. Su **objetivo principal es localizar y clasificar múltiples objetos simultáneamente** dentro de una imagen, proporcionando para cada uno una caja delimitadora, su categoría y un score de confianza.

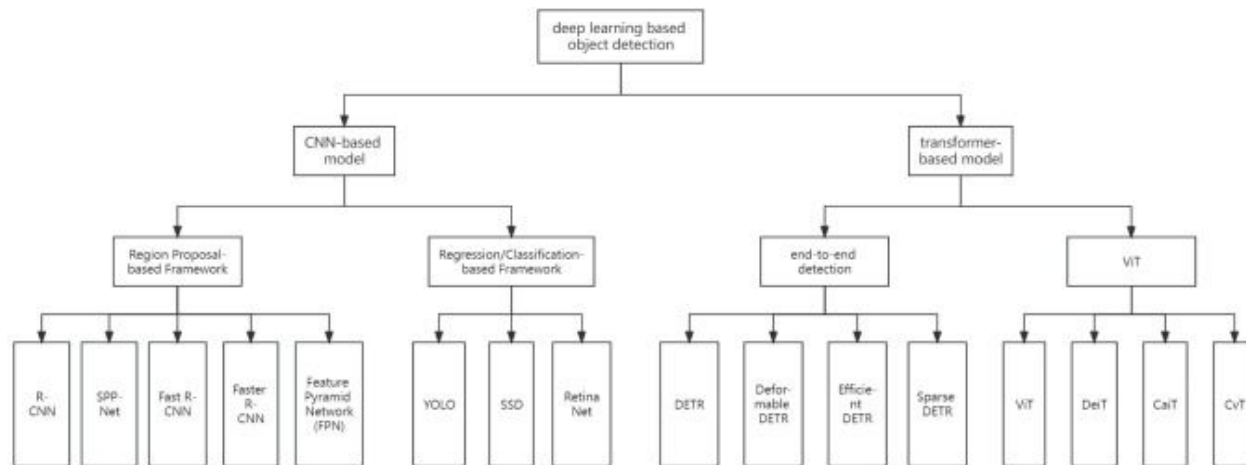
Diferencias clave:

- **Clasificación:** identifica qué hay en la imagen (una etiqueta global)
- **Detección:** localiza cada objeto con cajas y etiquetas múltiples
- **Segmentación:** delimita el contorno exacto píxel a píxel
- **Detección de anomalías:** hay algo raro en esta imagen



Tipos de arquitecturas de Object Detection

Hay muchos tipos de arquitecturas, estas son las familias más importantes



Evolución histórica de los detectores

Los modelos de detección de objetos han evolucionado significativamente en la última década, nos centraremos en tres grandes etapas arquitectónicas:

1 Two-Stage (2014-2016)

R-CNN, Fast R-CNN, Faster R-CNN: arquitecturas de dos etapas que primero proponen regiones candidatas y luego las clasifican. Precisos pero complejos y lentos.

2 One-Stage (2015→)

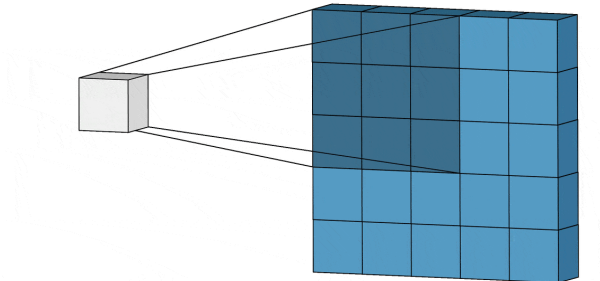
YOLO revoluciona el campo: una sola red neuronal procesa la imagen completa en tiempo real, predicando cajas y clases simultáneamente mediante una cuadrícula.

3 Transformers (2020→)

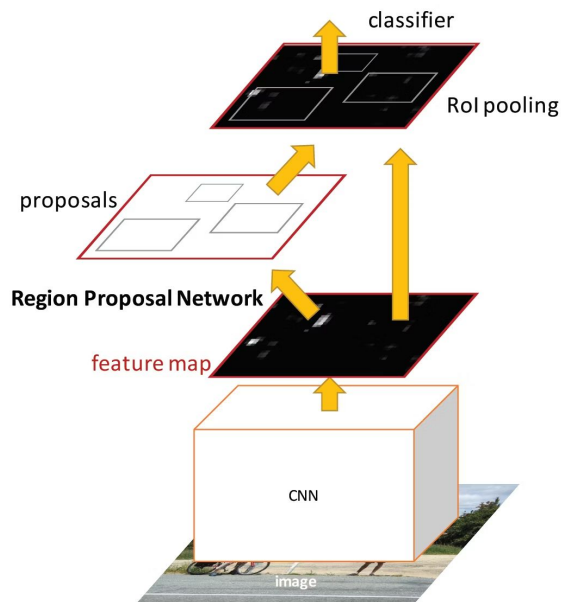
DETR y RF-DETR: pipelines limpios basados en atención, sin anchors predefinidos ni supresión de no-máximos (NMS). Arquitectura más simple y generalizable.

Redes neuronales convolucionales (CNNs)

Las Redes Neuronales Convolucionales (CNNs) son una parte muy importante de la visión artificial moderna. Están diseñadas para procesar datos visuales directamente, utilizando capas especializadas que detectan características como bordes, texturas y patrones en las imágenes de manera jerárquica.



Este enfoque permite a las CNNs aprender representaciones complejas de las imágenes, lo que las hace excepcionalmente potentes para estas tareas.



Two-Stage: Precisión a costa de complejidad

Los detectores de dos etapas, como la familia R-CNN, dividen el problema en **dos fases secuenciales**: primero generan propuestas de regiones candidatas (lugares donde podría haber objetos), y luego clasifican y refinan cada propuesta.

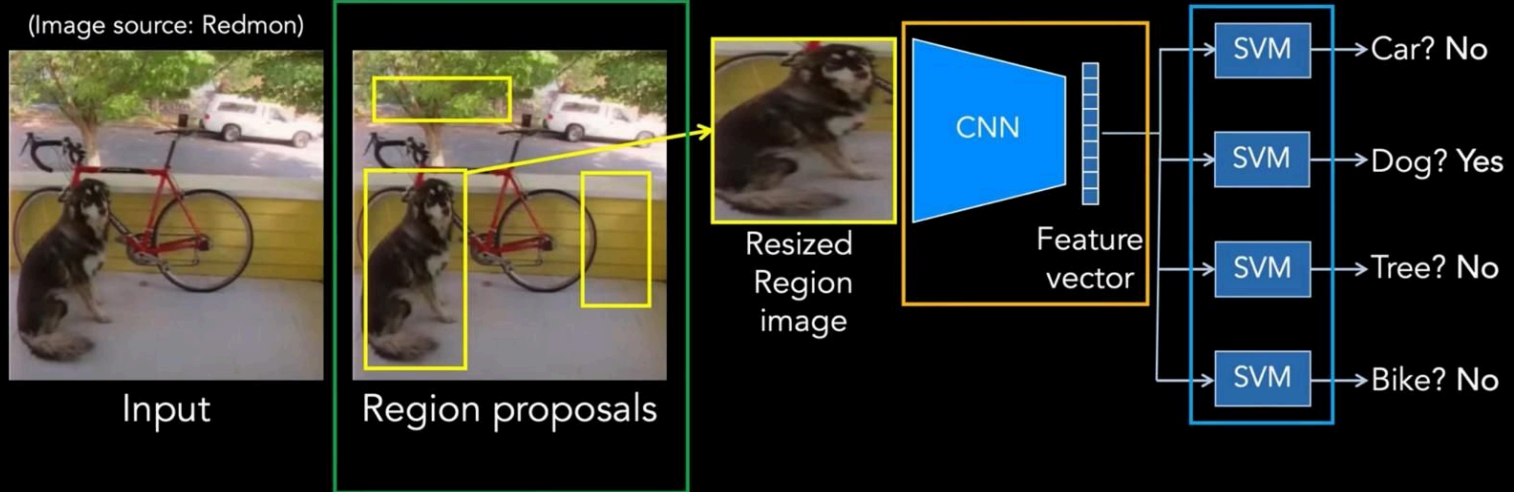
ROI Pooling acelera el proceso al extraer features de forma eficiente, pero el pipeline sigue siendo computacionalmente costoso. La generación inicial de regiones puede proponer cientos de candidatos, ralentizando la inferencia.

Trade-off característico: excelente precisión en benchmarks, pero latencia elevada para aplicaciones en tiempo real. Ideales cuando la exactitud es más importante que la velocidad. Difícil de optimizar

R-CNN

Region proposal + CNN + Clasificador

R-CNN (Region-based CNN)



Girshick *et. al* (2014)

"Rich feature hierarchies for accurate object detection and semantic segmentation"

Comparación de velocidad de RCNNs

Results

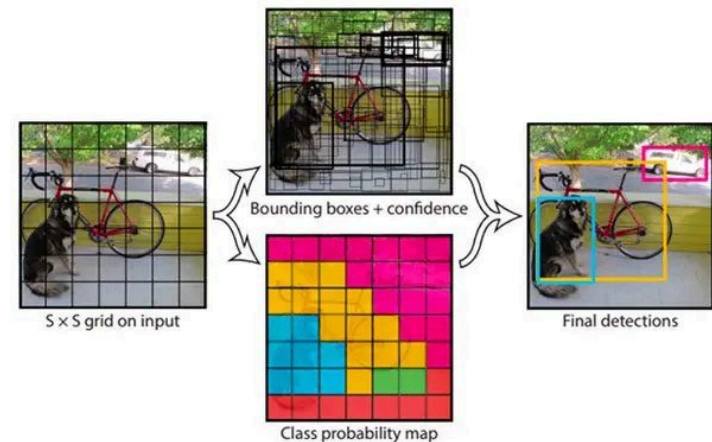
	RCNN	Fast RCNN	Faster RCNN
Proposal Generation Method	Selective Search	Selective Search	RPN
Proposal Generation Time	1500 ms	1500 ms	10 ms
Predictions for Generated Proposals Time / Image	47000 ms	320 ms	190 ms
Speedup Factor	X	25X	250X

YOLO: Detección como regresión directa

YOLO (You Only Look Once) reformula la detección como un **problema de regresión único**. La imagen se divide en una cuadrícula $S \times S$, donde cada celda predice directamente las cajas delimitadoras y las probabilidades de clase de los objetos cuyo centro cae en ella.

Ventaja clave: una única pasada hacia adelante por la red permite velocidades de procesamiento en tiempo real (30+ FPS), manteniendo precisión competitiva. Esta arquitectura one-stage elimina la necesidad de generar propuestas de región por separado.

- Cada celda predice múltiples cajas con sus scores de confianza
- Las predicciones ocurren simultáneamente para toda la imagen
- Ideal para aplicaciones que requieren latencia mínima



La Arquitectura YOLO: Velocidad y Simplicidad

Cómo el modelo procesa la imagen en una sola pasada ("You Only Look Once")

1. La estrategia de la cuadrícula (SxS Grid)

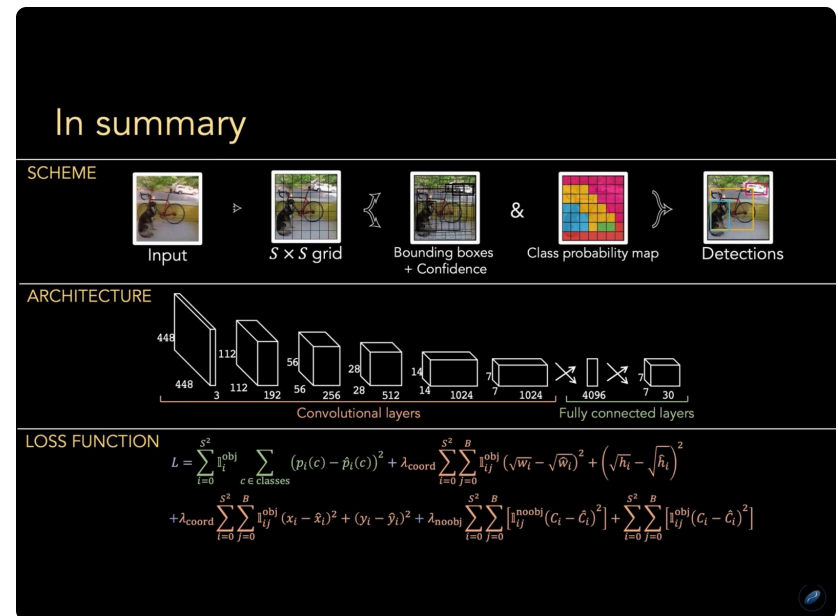
YOLO divide la imagen de entrada en una cuadrícula. Si el centro de un objeto cae en una celda específica, esa celda es la responsable de detectarlo.

2. Predicción simultánea (Bounding Boxes + Clases)

Cada celda predice recuadros (bounding boxes) con su ubicación, tamaño y nivel de confianza, y la categoría del objeto (mapa de probabilidad), todo al mismo tiempo.

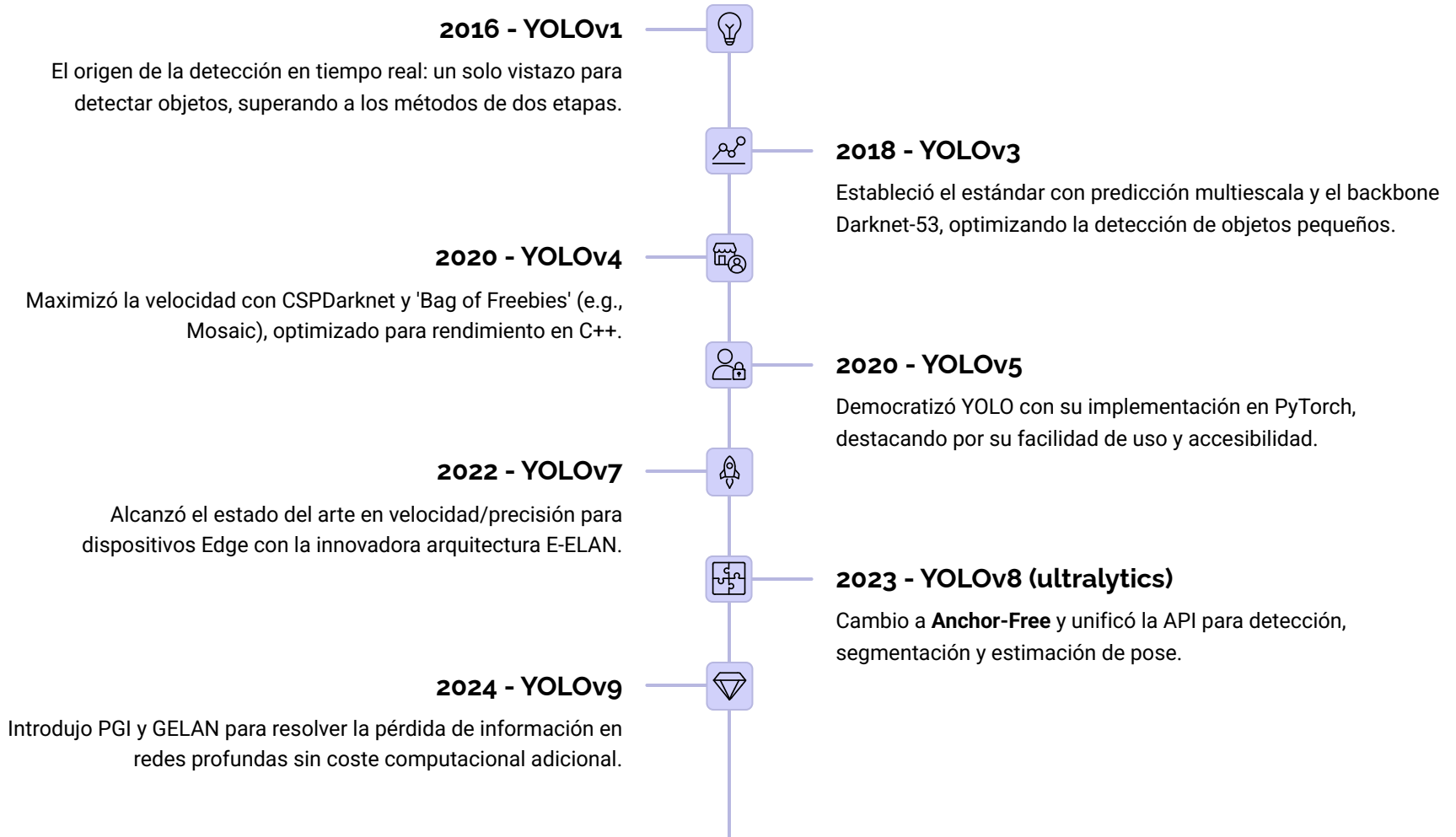
3. Arquitectura unificada (CNN)

Una única Red Neuronal Convolucional (CNN) procesa la imagen completa, conectando directamente los píxeles de entrada con las coordenadas y clases de salida, eliminando etapas intermedias complejas.



Evolución de la familia YOLO (2016-2024)

La serie YOLO ha transformado la detección de objetos, pasando de un concepto novedoso a un ecosistema robusto y versátil.



Por qué las CNNs no son suficientes

Aunque las CNNs revolucionaron la visión artificial, presentan limitaciones arquitectónicas que restringen su capacidad para tareas más complejas, especialmente en escenarios con objetos distantes o relaciones espaciales no evidentes.

Campo receptivo limitado

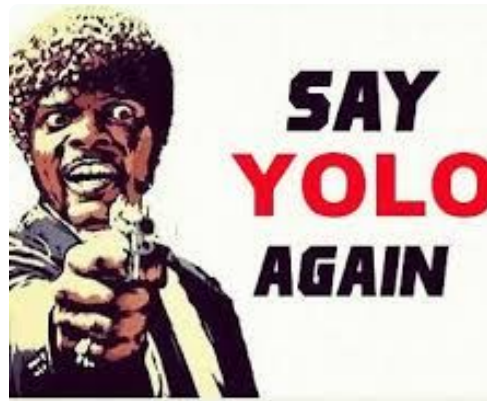
Las CNNs procesan información en parches locales. Para entender relaciones a larga distancia, necesitan muchas capas, lo que aumenta la profundidad del modelo y dificulta la propagación de gradientes. Esto es ineficiente para capturar el contexto global de una imagen.

Sesgo inductivo fuerte

Asumen una fuerte localidad espacial e invarianza a traslación. Esto es útil para objetos que aparecen en cualquier lugar de la imagen, pero dificulta la modelización de relaciones abstractas o estructuras que no se ajustan a patrones de vecindad directa.

Componentes Ad-Hoc

Dependen de elementos manuales y heurísticos como los "anchors" predefinidos (cajas de diferente tamaño y aspecto) y el proceso de Supresión de No Máximos (NMS) para eliminar redundancia. Estos componentes añaden complejidad, hiperparámetros y son difíciles de optimizar globalmente.

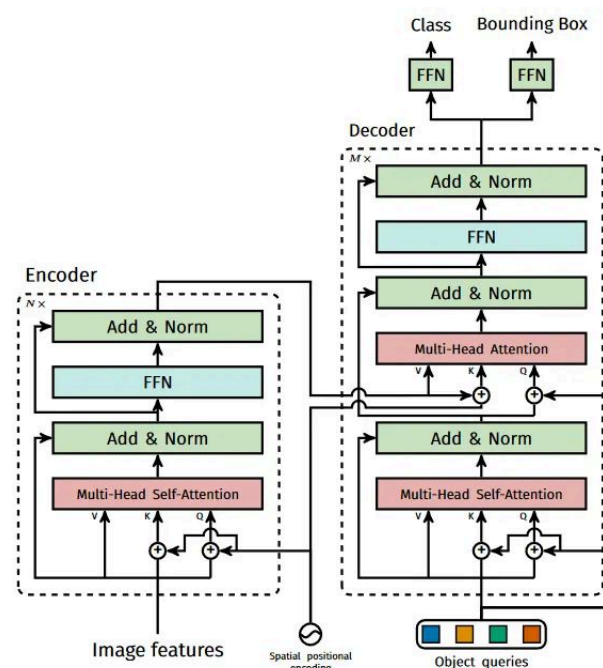


DETR: Transformers para detección

DETR (DEtection TRansformer) introduce un **paradigma completamente nuevo**: utiliza una arquitectura encoder-decoder con "object queries" aprendibles que predicen un conjunto fijo de objetos directamente.

Pipeline simplificado: Un backbone (como ResNet) extrae features visuales, el encoder Transformer las procesa globalmente, y el decoder utiliza N queries (típicamente 100) para predecir N potenciales objetos, cada uno con su caja y clase.

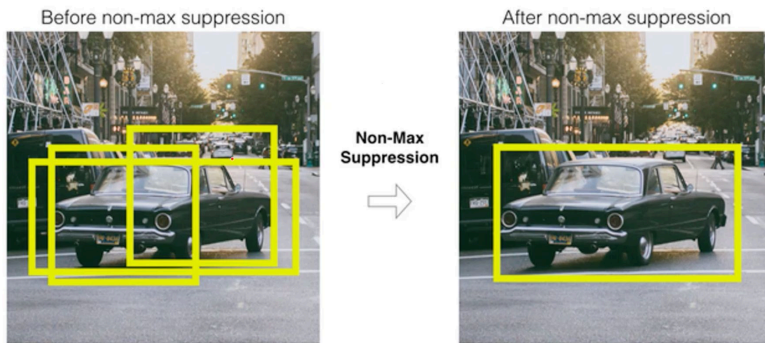
- Sin hiperparámetros de NMS que ajustar
- Sin anchors dependientes del dataset
- Arquitectura end-to-end diferenciable y limpia



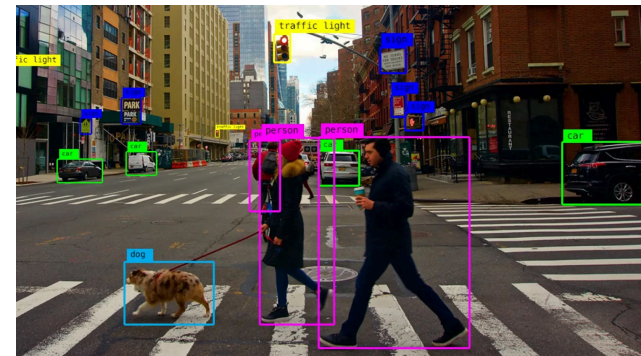
Fin del NMS: matching húngaro

Una de las mayores innovaciones de DETR es la eliminación del proceso de Supresión de No Máximos (NMS) gracias al matching húngaro, simplificando radicalmente el pipeline de detección, un algoritmo de asignación bipartita que encuentra la correspondencia óptima entre predicciones y objetos reales durante el entrenamiento. Esto elimina la necesidad de anchors predefinidos y supresión de no-máximos (NMS).

YOLO (pre-NMS): predicciones redundantes



DETR: predicciones directas y únicas



- Predice miles de cajas candidatas y necesita un proceso de filtrado (NMS) para eliminar las duplicadas.
- Asigna una predicción única a cada objeto real utilizando un algoritmo de matching húngaro basado en un coste global.

RF-DETR

RF-DETR representa un avance significativo al integrar eficientemente la potencia de los Transformers con innovaciones clave, superando a menudo a otros modelos en velocidad y precisión. Su diseño permite una detección de objetos en tiempo real, adaptable a diversas tareas.



Backbone DINOv2 pre-entrenado

Utiliza un potente vision transformer pre-entrenado para un reconocimiento robusto de patrones visuales, lo que facilita una rápida adaptación y reduce la necesidad de grandes volúmenes de datos para nuevas tareas.



Atención deformable eficiente

A diferencia de la autoatención estándar, este mecanismo se centra en un pequeño conjunto de puntos de muestreo clave, permitiendo procesar mapas de características de alta resolución de manera eficiente y mejorando la detección de objetos pequeños.

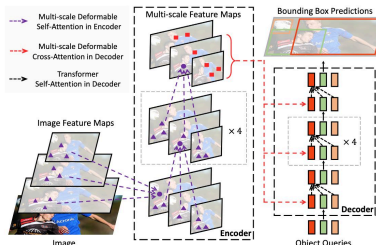


Pipeline modular y sin anchors

Su diseño flexible elimina la necesidad de anchors predefinidos y la supresión de no-máximos (NMS), simplificando la arquitectura y haciéndola más generalizable y fácil de optimizar para distintas aplicaciones.

RF-DETR

El funcionamiento de RF-DETR se basa en una arquitectura de transformer diseñada para la detección de objetos, procesando la información visual a través de una serie de pasos secuenciales y refinamientos.



01

Combinación de features multi-escala

RF-DETR integra features provenientes de diferentes escalas de resolución de la imagen. Esto se realiza antes de que los datos lleguen al encoder del Transformer, asegurando una comprensión contextual rica desde el principio.

02

Alineación con atención deformable en el encoder

El encoder del Transformer procesa estas features multiescala y utiliza un mecanismo de atención deformable. Este enfoque permite alinear y enfocar eficientemente las características más relevantes, ajustándose dinámicamente.

03

Interacción de queries con Cross-Attention

Las "object queries" interactúan con los niveles de features del encoder a través de un mecanismo de cross-attention. Esto permite que cada query acceda a información relevante de diversas escalas para proponer ubicaciones.

04

Refinamiento de predicciones en el decoder

El decoder de RF-DETR consta de múltiples capas que refinan iterativamente las predicciones de los objetos. Cada capa toma las predicciones de la anterior y las mejora.

05

Salida final de bounding boxes

Finalmente, el modelo genera directamente las bounding boxes para cada objeto, sin la necesidad de post-procesamiento como NMS, lo que agiliza el proceso de detección.

Por qué NO usamos ultralytics en clase

Ultralytics, a pesar de su popularidad, presenta varias características que lo hacen inadecuado para el entorno académico y de investigación que buscamos promover, especialmente porque queremos fomentar el uso de código verdaderamente open source, transparente y con licencias que no limiten la investigación ni el desarrollo industrial.



Licencia problemática

La combinación de licencias AGPL y comercial fuerza a los usuarios a liberar su código o a pagar, lo que contradice el espíritu de un verdadero open-source y limita su uso comercial.



Apropiación de marca YOLO

La designación de sus versiones (YOLOv5-YOLOv11) como "YOLO oficial" es engañosa, ya que no provienen de la familia original de Joseph Redmon ni de la línea académica, causando desinformación generalizada.



Código no modular

El código está fuertemente orientado a su interfaz de línea de comandos (CLI), lo que dificulta su estudio, comprensión interna y adaptación para fines de investigación o proyectos más personalizados.



Limitaciones académicas

Su enfoque lo hace poco apto para la enseñanza y la investigación seria. La falta de modularidad y la escasa reproducibilidad científica lo alejan de los estándares de los papers académicos.

Por qué SÍ usamos RF-DETR en clase

RF-DETR es la base de nuestro currículo de detección de objetos por su diseño avanzado, su alineación con los principios de la investigación y la enseñanza moderna en visión artificial, y porque es un proyecto completamente open source bajo licencia Apache 2.0.



Licencia apache 2.0

Libre para industria, investigación y enseñanza. Sin obligaciones ocultas que limiten la innovación o el despliegue de proyectos.



Código limpio y modular

Diseñado para ser fácil de leer, modificar y explicar a alumnos, facilitando la experimentación y el aprendizaje de la implementación práctica.



Arquitectura moderna y clara

Basado en la familia DETR, ofrece una comprensión profunda de cómo funciona un detector de objetos de vanguardia, sin la complejidad de sistemas heredados.



Muy buen rendimiento

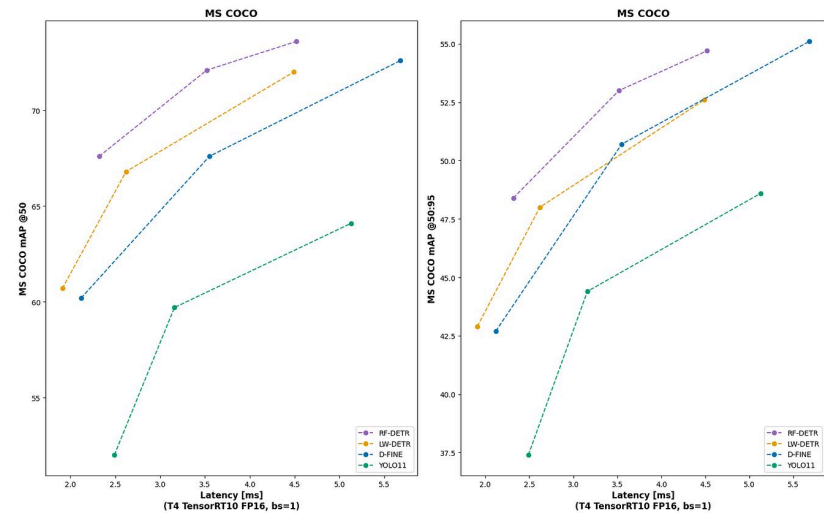
Ofrece una velocidad de inferencia y entrenamiento superior a muchos modelos YOLO modernos, optimizando los tiempos de desarrollo y prueba en el aula.

RF-DETR vs YOLO en producción

La elección entre RF-DETR y YOLO depende de tus **prioridades específicas de producción**.

Consideraciones prácticas:

- **Licencia:** YOLO suele usar AGPL (obliga a publicar código modificado); RF-DETR usa Apache 2.0 (más permisiva)
- **Mantenibilidad:** RF-DETR evita el ajuste manual de anchors y thresholds de NMS
- **Adopción:** YOLO tiene ecosistema más maduro y extenso

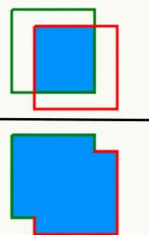


IoU (Intersection over Union)

La **Intersection over Union (IoU)** es una métrica fundamental que mide el solapamiento entre dos cajas delimitadoras: una caja predicha por nuestro modelo y una caja de verdad (ground truth). Esta métrica es crucial para determinar si una predicción es correcta (True Positive) o no (False Positive), y su valor se encuentra en un rango de 0 (sin solapamiento) a 1 (solapamiento perfecto).

Para visualizarlo, considera una caja delimitadora predicha por el modelo (en rojo) y la caja de verdad que marca la ubicación real de un objeto (en verde). La IoU calcula el área donde ambas cajas se superponen, dividida por el área total que ocupan ambas cajas combinadas.

$$\text{IoU} = \frac{\text{area of overlap}}{\text{area of union}}$$



Umbrales típicos de IoU

- **IoU > 0.5:** Es el umbral común para considerar una detección como True Positive (TP), especialmente utilizado en el conjunto de datos COCO.
- **IoU > 0.75:** Un umbral más estricto que se usa para evaluar la alta precisión de las detecciones.
- **IoU < umbral:** Cualquier detección con un IoU inferior al umbral establecido se clasifica como False Positive (FP).

¿Por qué es importante?

- Sin IoU, no podríamos distinguir eficazmente entre True Positives (detecciones correctas) y False Positives (falsas alarmas).
- Es la base para el cálculo de métricas más complejas como la Precisión, el Recall, la Average Precision (AP) y la mean Average Precision (mAP).
- Determina directamente la calidad y la exactitud espacial de las detecciones de objetos de nuestro modelo.

Fundamentos: TP, FP, FN (y por qué no hay TN)



TP (True positives)

Detecciones correctas: cajas predichas que coinciden con objetos reales ($IoU > \text{umbral}$)



FP (False positives)

Falsas alarmas: cajas predichas donde no hay objeto real o con IoU muy bajo



FN (False negatives)

Objetos perdidos: objetos reales que el modelo no detectó

Por qué NO existen True Negatives (TN) en object detection

La métrica de True Negatives, común en problemas de clasificación binaria, no aplica de la misma manera en la detección de objetos:

- **En clasificación binaria:** TN = "predice que no hay gato y efectivamente no hay gato".
- **En detección de objetos:** El modelo predice cajas delimitadoras en ubicaciones específicas donde cree que hay un objeto.
- No existe el concepto explícito de "predecir ausencia de objeto en una ubicación".
- Las ubicaciones donde no se predice nada son infinitas e impracticables de medir.

Por esta razón, las métricas en detección de objetos se enfocan únicamente en **TP, FP y FN**, ya que son los resultados relevantes de las predicciones del modelo.

Métricas de evaluación: precision y recall

En la detección de objetos, entender la calidad de las predicciones va más allá de solo saber si se detectó algo. Precision y recall nos dan una visión completa de la fiabilidad y exhaustividad del modelo.

La importancia de la precisión

Mide la **confiabilidad de las detecciones positivas**. Un detector con alta precisión minimiza los falsos positivos (detectar algo que no está ahí o identificarlo incorrectamente). Es crucial en aplicaciones donde un error de detección tiene costes elevados, como en inspección de calidad o sistemas de seguridad.

$$Precision = \frac{TP}{TP + FP}$$

Esto responde a la pregunta: "De todas mis predicciones, ¿cuántas son correctas?"

La importancia del recall

Mide la **capacidad del modelo para encontrar todos los objetos relevantes** en una imagen. Un alto recall significa pocos falsos negativos (no detectar un objeto que sí está presente). Es vital en escenarios donde es crítico no omitir ningún objeto, como la detección de personas en vigilancia o tumores en imágenes médicas.

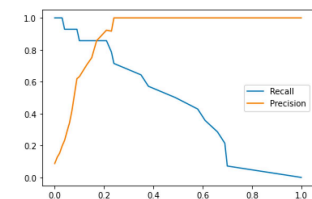
$$Recall = \frac{TP}{TP + FN}$$

Esto responde a la pregunta: "De todos los objetos reales, ¿cuántos detecté?"

Ejemplos prácticos de interpretación:

- Si un modelo tiene una precision alta (por ejemplo, 0.9), significa que el 90% de las detecciones que realiza son correctas. Es muy fiable en lo que afirma haber encontrado.
- Si un modelo tiene un Recall alto (por ejemplo, 0.8), significa que detectó el 80% de todos los objetos que realmente estaban presentes en las imágenes. Es bueno para no perder objetos.

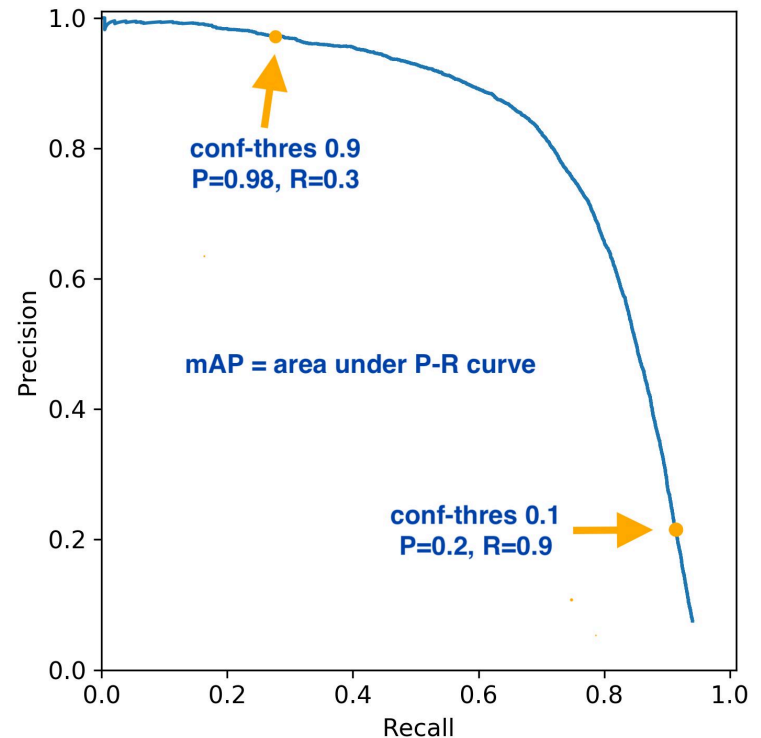
Es importante entender que Precision y Recall a menudo presentan un **trade-off**. Mejorar una métrica a menudo puede llevar a la disminución de la otra. Por ejemplo, si un modelo es muy agresivo en sus detecciones para asegurar que no se pierda nada (alto Recall), podría también generar más falsas alarmas (baja Precision). La elección del equilibrio adecuado depende de los requisitos específicos de la aplicación.



Métricas de evaluación: AP y mAP

AP (Average Precision): Representa el área bajo la curva Precision-Recall. Mide la habilidad del modelo para detectar todos los objetos relevantes (recall alto) mientras mantiene un número bajo de falsos positivos (precisión alta). Un valor de AP más alto indica un mejor rendimiento.

mAP (mean Average Precision): Es la métrica estándar para evaluar detectores de objetos. Se calcula promediando los valores de AP para todas las clases de objetos detectadas, y a menudo, en diferentes umbrales de IoU (ej., mAP@0.5 o mAP@[0.5:0.95]).



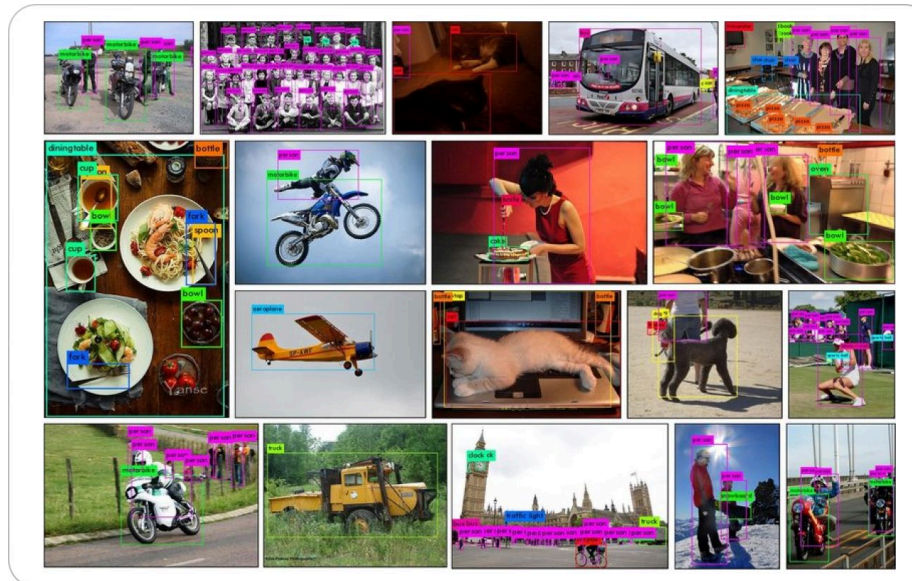
Límites prácticos: COCO y el mundo real

El dataset COCO, con sus 80 clases estándar (persona, coche, perro, silla...), es excelente para benchmarking pero normalmente **no cubre las necesidades reales de industria o dominios especializados**.

Problema del dominio: Una fábrica necesita detectar "tornillo oxidado", "conector suelto", "clip roto" o "soldadura defectuosa". Estos objetos específicos no existen en COCO.

Coste de adaptación: Re-anotar miles de imágenes para nuevas categorías es caro y lento. Entrenar desde cero o fine-tunear requiere tiempo de GPU, experimentación y validación.

Esta limitación motiva la necesidad de **métodos más flexibles**: detectores guiados por texto (zero-shot), modelos fundacionales multimodales, y técnicas que puedan adaptarse rápidamente a nuevos dominios sin anotación exhaustiva.



Buenas prácticas al evaluar detectores

Evaluar detectores va más allá de mirar mAP en COCO. Para deployment real, debes:

- 1. Medir en tu dominio específico:** Crea un conjunto de validación representativo de tus datos reales. Un modelo con 60% mAP en COCO puede tener 40% o 80% en tu caso de uso particular.
- 2. Balancear velocidad vs precisión:** Un detector con 2% más de AP pero 50% más lento puede no valer la pena si necesitas procesar 30 FPS. Mide latencia end-to-end, no solo teóricos.
- 3. Analizar errores cualitativamente:** Inspecciona visualmente los casos de:
 - **Falsos Positivos (FP):** detecciones donde no hay objeto
 - **Falsos Negativos (FN):** objetos que el modelo no detectó
 - **Casos límite:** objetos parcialmente ocluidos, muy pequeños, o en condiciones de iluminación extrema

Los patrones de error te dirán si necesitas más datos, data augmentation específico, o cambiar de arquitectura.

El siguiente paso

La evolución de la Visión Artificial nos ayuda a superar limitaciones clave de estos modelos, abriendo paso a nuevas arquitecturas más flexibles y potentes.



Limitaciones actuales

- Solo 80 clases COCO
- No entienden lenguaje
- Requieren reentrenamiento



Modelos multimodales

- Entienden texto + imagen
- Zero-shot con lenguaje natural
- Sin reentrenamiento constante



Hugging Face

2. El ecosistema de modelos de IA

Buscar, evaluar licencias y usar modelos fácilmente

Hugging Face se ha convertido en el **repositorio central de facto** para modelos de IA modernos, datasets, demos interactivas y herramientas de desarrollo. Su ecosistema integrado permite descubrir, probar y desplegar modelos.

El ecosistema Hugging Face

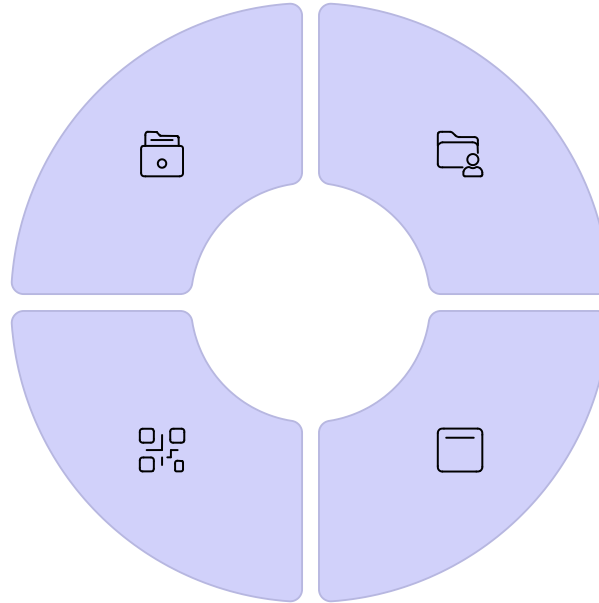
Hugging Face no es solo un repositorio, es un **ecosistema completo de cuatro pilares interconectados**:

Model Hub

Catálogo con más de 500.000 modelos preentrenados para todas las tareas imaginables: visión, NLP, audio, multimodal.

Transformers

Librería Python que unifica la API de carga, inferencia y fine-tuning para miles de arquitecturas diferentes.



Datasets

Más de 100.000 datasets públicos listos para entrenar y evaluar, con APIs unificadas de carga y procesamiento.

Spaces

Demos web interactivas con Gradio o Streamlit, desplegadas sin servidor. Prueba modelos en segundos con interfaz gráfica.

El flujo típico es: **buscar modelo** → **probar en Space** → **descargar con Transformers** → **integrar en tu aplicación**.



huggingface

Models – Hugging Face

Explore machine learning models.



Cómo buscar modelos efectivamente

El Model Hub permite **filtrar por múltiples criterios** para encontrar exactamente lo que necesitas:

- **Tarea:** object-detection, image-classification, text-generation, etc.
- **Licencia:** Apache-2.0, MIT, CreativeML, Research-only
- **Popularidad:** número de descargas, likes, fecha de última actualización

La Model Card es tu mejor amiga. Cada modelo debe incluir:

Descripción y uso

Para qué fue entrenado, qué tareas resuelve, cómo usarlo con código de ejemplo.

Limitaciones

Sesgos conocidos, casos donde falla, advertencias de seguridad y aspectos éticos.

Licencia

Términos legales de uso: ¿puedes usarlo comercialmente? ¿Debes atribuir autoría?

Archivos

Configuración JSON, pesos del modelo (.safetensors preferido), tokenizers, ejemplos.

Métricas

Resultados en benchmarks estándar, comparación con otros modelos.

Licencias: evita problemas legales

La licencia del modelo determina **cómo puedes usarlo legalmente**. Ignorar este aspecto puede causar problemas serios en producción.

✓ Comercial OK

- **Apache-2.0:** muy permisiva, ideal para empresas
- **MIT:** similar a Apache, aún más simple
- **BSD:** variantes 2-clause y 3-clause, ambas permisivas

⚠ Con restricciones

- **OpenRAIL/CreativeML:** uso comercial OK, pero con cláusulas de uso responsable
- **CC BY-SA:** requiere compartir derivados bajo la misma licencia

✗ No comercial

- **Research Only:** solo investigación académica
- **CC BY-NC:** prohibido uso comercial

❗ **Caso especial AGPL:** Algunas familias YOLO populares usan AGPL, que **obliga a publicar tu código fuente completo** si integras el modelo en una aplicación web o servicio. Si no quieres publicar tu código, busca alternativas con Apache-2.0 como RT-DETR o RF-DETR.

Usar modelos

La librería Transformers hace que cargar y usar modelos sea **muy sencillo**. El modelo mental clave:

01

Hub remoto

El modelo vive en Hugging Face Hub identificado por su nombre (ej: "facebook/detr-resnet-50").

03

Carga en memoria

El modelo se instancia en RAM/VRAM según el dispositivo (CPU/GPU) que especifiques.

02

Descarga a caché

Primera ejecución: descarga config.json, pesos .safetensors, y metadatos a ~/.cache/huggingface/

04

Inferencia

Procesas inputs y obtienes predicciones. Puedes usar GPU y mixed precision para acelerar.

Buenas prácticas:

- **Seguridad:** Prefiere .safetensors sobre .bin (formato más seguro)
- **Offline:** Predescarga con snapshot_download, controla cache_dir
- **Rendimiento:** Usa float16 y device="cuda" cuando tengas GPU
- **Pipelines:** Para pruebas rápidas, usa pipeline("task"); en producción, clases específicas dan más control

Inferencia en 3 líneas de código

```
from transformers import pipeline

# 1. Cargar modelo (automáticamente baja config y pesos)
detector = pipeline("object-detection", model="facebook/rt-detr-l")

# 2. Inferencia
results = detector("imagen.jpg")

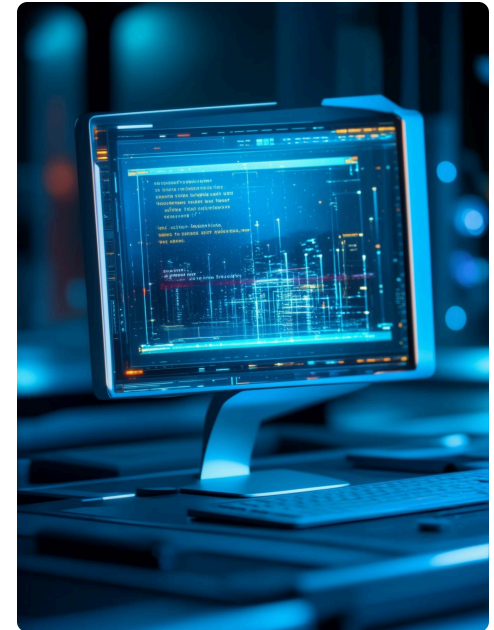
# 3. Resultado
print(results) # [{'label': 'cat', 'score': 0.99, 'box': {...}]}
```

📄 La librería `transformers` maneja la descarga, el pre-procesamiento y la inferencia automáticamente.

3. Multimodalidad: unir visión y lenguaje

CLIP · BLIP · Grounding DINO · VLM (Qwen2.5-VL)

Los modelos fundacionales multimodales representan un **salto cualitativo**: combinan visión y lenguaje para entender instrucciones en texto natural, describir imágenes, detectar objetos mediante descripciones verbales, y razonar sobre contenido visual.



Modelos Fundacionales



Definición

Son modelos preentrenados a gran escala que aprenden representaciones generales y transferibles.



Importancia

Capturan patrones robustos gracias al preentrenamiento masivo y funcionan bien incluso en zero-shot.



Capacidades

Permiten reutilizar el conocimiento en muchas tareas, reducen anotación y evitan entrenar desde cero.



Ejemplos

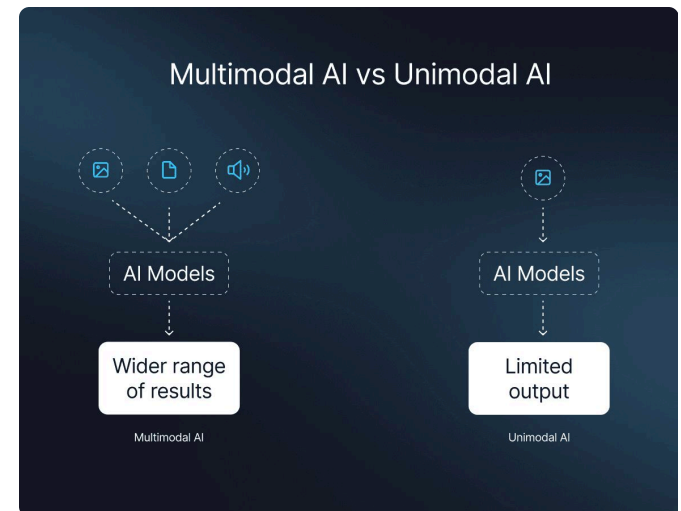
- CLIP: 400 millones de pares imagen-texto.
- DINOv2: 142 millones de imágenes (sin etiquetas)
- SAM: 11 millones de imágenes + (1.000 M) de máscaras (segmentaciones)

Multimodalidad

La multimodalidad añade una dimensión importante: estos modelos procesan imagen + texto simultáneamente, aprendiendo correspondencias entre ambos dominios. Esto les permite entender instrucciones verbales sobre contenido visual y generar descripciones textuales de imágenes.

Ejemplos de modelos multimodales:

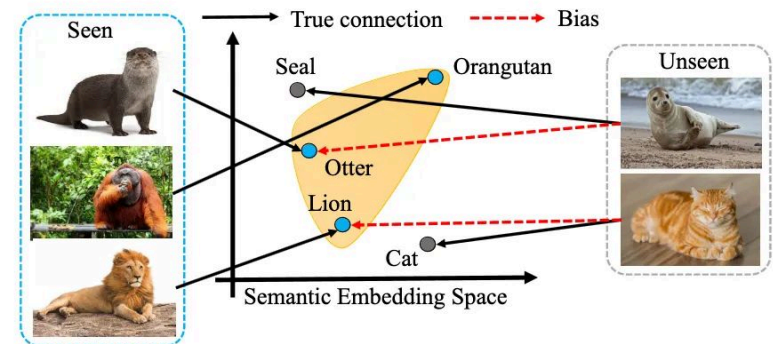
- **CLIP:** Relaciona imágenes y texto en un espacio compartido
- **BLIP:** Genera descripciones y responde preguntas sobre imágenes
- **Grounding DINO:** Detecta objetos mediante descripciones textuales
- **VLM (Qwen2.5-VL):** Conversa y razona sobre contenido visual



Zero-Shot y composicionalidad

La capacidad **zero-shot** significa que el modelo puede resolver tareas nuevas sin entrenamiento específico para ellas. La clave está en la **composicionalidad**: el modelo aprende conceptos atómicos ("perro", "rojo", "corriendo") y los combina libremente.

Ejemplo práctico: El modelo solo ha visto **nutria, orangután y león**. En una imagen nueva, **foca y gato** (nunca vistos) se proyectan en el **espacio semántico** cerca de los conceptos aprendidos (foca≈nutria, gato≈león). Así puede **reconocer clases nuevas sin re-entrenar**, añadiendo categorías al instante y cubriendo con coste de anotación casi cero.



Impacto en producción:

- Se ahorran miles de horas de anotación manual para nuevas variantes
- Se puede experimentar en minutos en lugar de semanas

Esta flexibilidad es revolucionaria para dominios industriales o médicos donde las categorías cambian constantemente.

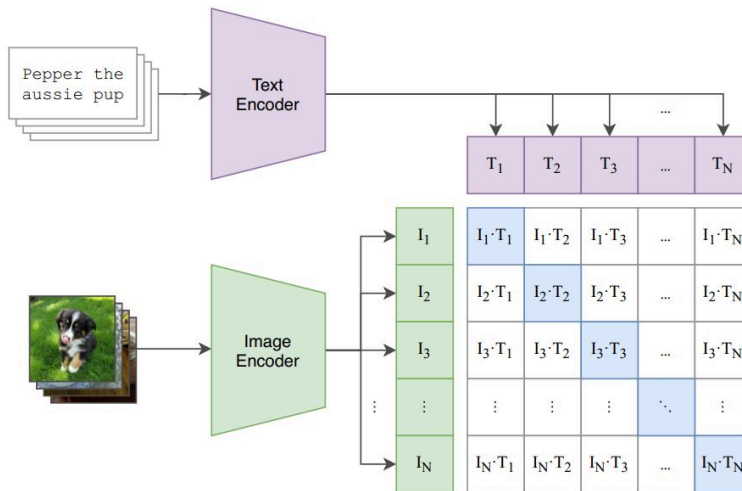
CLIP: imagen - texto (Retrieval/Zero-Shot)

CLIP (Contrastive Language-Image Pre-training) aprende un **espacio de embeddings compartido** donde imágenes y textos similares están cerca y diferentes están lejos.

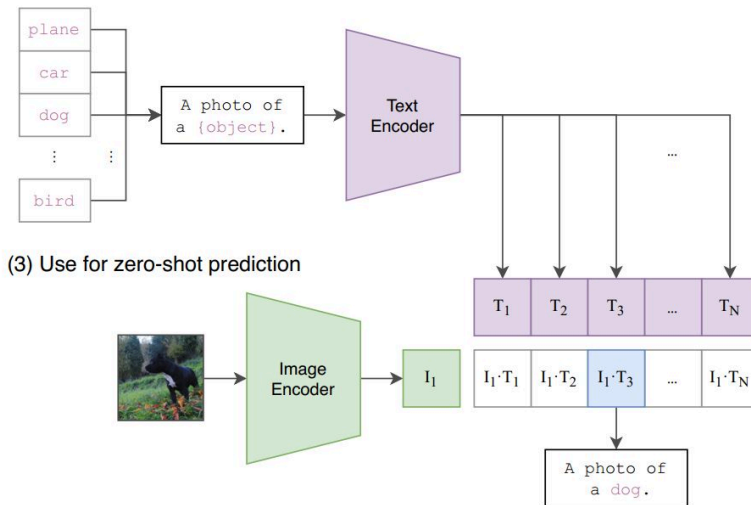
Arquitectura: Un encoder de imagen y un encoder de texto procesan sus respectivas entradas en paralelo, proyectándolas al mismo espacio vectorial. La similitud se mide con producto escalar o coseno.

- **Clasificación zero-shot:** "¿Es esta imagen un gato o un perro?" → calculas similitud con ambos textos
- **Búsqueda por texto:** "encuentra imágenes de tornillos oxidados" en un catálogo sin etiquetar
- **Filtrado rápido:** CLIP es ligero y rápido, ideal como primera etapa antes de modelos más pesados

(1) Contrastive pre-training



(2) Create dataset classifier from label text



(3) Use for zero-shot prediction

Cómo funciona CLIP

CLIP relaciona imágenes y texto mediante un **entrenamiento contrastivo (contrastive learning)** y luego aplica ese conocimiento para tareas como la **clasificación zero-shot**.



Imagen a vector

La imagen se convierte en un vector (embedding) con el Image Encoder.



Textos a vectores

Los textos de las posibles clases se convierten en vectores con el Text Encoder.



Cálculo de similitud

Se calcula la similitud entre el vector de la imagen y cada vector de texto.

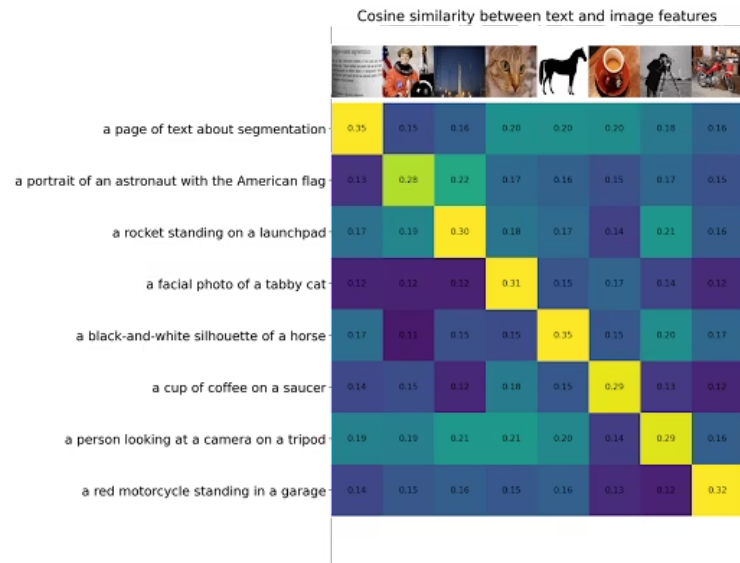
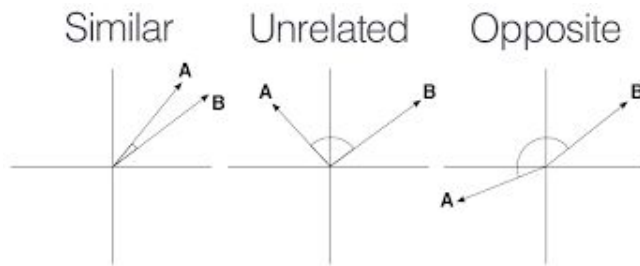


Predicción

La clase con mayor similitud es la predicción final del modelo.

Ejemplo:

- **Imagen:** Foto de un gato
- **Textos:** "una astronauta...", "un gato...", "un caballo..."
- **Similitudes:** [0.1, 0.9, 0.2]
- **Predicción:** "un gato..."



Entrenamiento de CLIP: Aprendiendo por asociación

Entrenamiento contrastivo a gran escala (400M de pares imagen-texto)

1

La entrada: Arquitectura de dos torres

CLIP utiliza dos codificadores independientes que trabajan en paralelo, proyectando la información a un espacio compartido:

- **Image Encoder:** (Vision Transformer o ResNet) convierte la imagen en un vector numérico (embedding).
- **Text Encoder:** (Un Transformer) convierte la descripción textual en otro vector numérico.

2

El proceso: La matriz de $N \times N$

El modelo recibe un lote (batch) de N imágenes y sus N textos correspondientes, creando una matriz de $N \times N$ posibles parejas.

En un lote de 32.000 pares, se generan 32.000 parejas correctas y millones de combinaciones incorrectas.

3

El objetivo: Maximizar la diagonal

El modelo se entrena con una única regla simple (**Contrastive Loss**):

- Aumentar la similitud (producto escalar) de los pares correctos (en la diagonal).
- Disminuir la similitud de todas las combinaciones incorrectas (fuera de la diagonal).

Limitaciones de CLIP

Qué NO puede hacer:

- Generar texto (solo embeddings)
- Detectar objetos (no proporciona bounding boxes)
- Segmentar imágenes (no produce máscaras)
- Responder preguntas sobre contenido visual
- Contar objetos con precisión

Problemas conocidos:

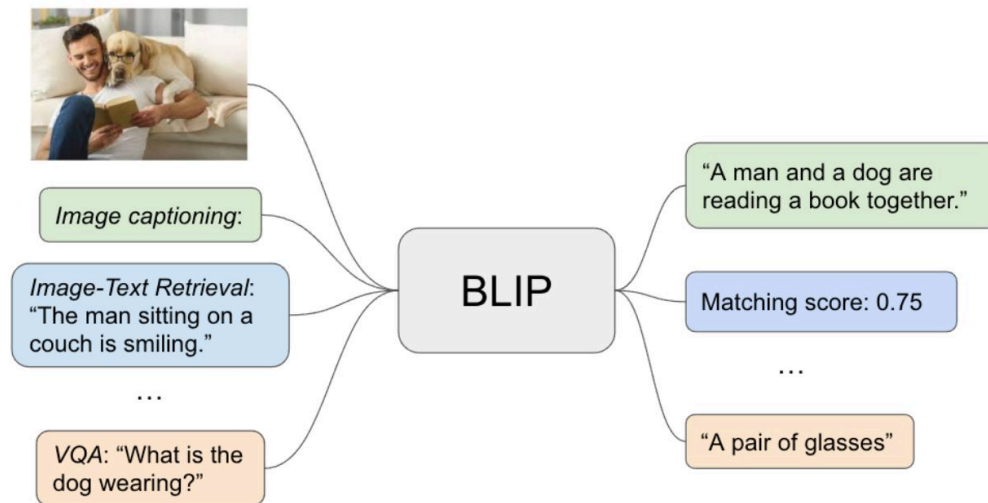
- Mal rendimiento con texto dentro de imágenes (OCR)
- Sesgos heredados del dataset de entrenamiento
- Confunde atributos visuales muy finos
- Poca comprensión de negaciones en el prompt
- Alta sensibilidad a la formulación del prompt

BLIP: describe y responde (generativo)

BLIP (Bootstrapping Language-Image Pre-training) va más allá de CLIP: no solo relaciona imágenes con texto, sino que **genera descripciones textuales** (captioning) y **responde preguntas** sobre el contenido visual (VQA).

Arquitectura: Combina un encoder de imagen con un decoder de lenguaje basado en Transformers. La fusión multimodal ocurre mediante capas de cross-attention que permiten al texto "mirar" las features visuales.

- **Documentación automática:** generar descripciones de productos, escenas industriales o imágenes médicas
- **Accesibilidad:** proporcionar descripciones textuales para personas con discapacidad visual
- **VQA simple:** "¿Qué color tiene el casco?" → "Amarillo", "¿Cuántas personas aparecen?" → "Dos"



Grounding DINO: texto → cajas (Open-Vocabulary)

Grounding DINO combina la potencia de DINO (detección) con la comprensión lingüística para realizar **detección guiada por lenguaje natural** sin clases fijas.

Cómo funciona: Le das un prompt textual ("encuentra todos los tornillos oxidados" o "localiza cascos amarillos") y el modelo devuelve bounding boxes para cada coincidencia, sin haber sido entrenado explícitamente en esas categorías.

- Elimina la necesidad de re-entrenar para cada nueva categoría de objeto
- Permite detección ad-hoc: cambias el texto y cambias lo que detectas
- Auto-labeling

Capacidades Avanzadas:

- **Detección compositiva:** "coches rojos y personas con cascos" → entiende atributos y combinaciones
- **Referring expressions:** "la persona alta de la izquierda" → localización específica mediante descripciones

Cómo funciona grounding DINO

Grounding DINO integra procesamiento de lenguaje e imagen en un pipeline unificado, permitiendo una detección de objetos altamente flexible y adaptada a descripciones textuales.



Codificación de texto

El prompt textual ("coche rojo y persona") se transforma en **features lingüísticas** ricas en significado a través de un modelo BERT.



Codificación de imagen

La imagen de entrada es procesada por un Swin Transformer para extraer **features visuales** de alta resolución.



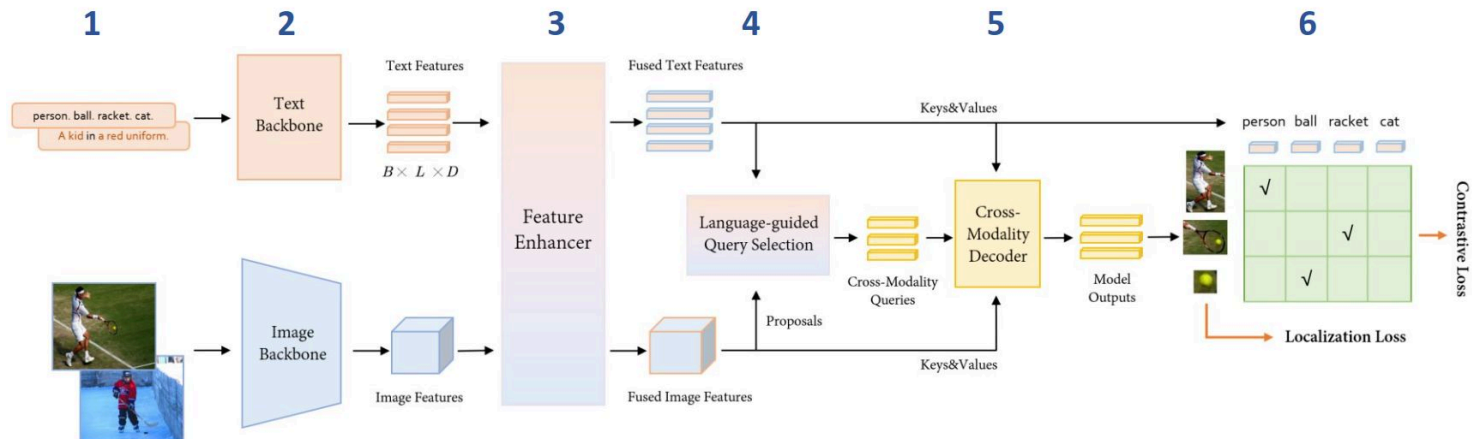
Fusión de características

Mediante **atención cruzada intermodal**, las features de texto guían la interpretación visual, alineando el lenguaje con las regiones espaciales relevantes de la imagen.



Detección head

Un decoder Transformer usa estas features fusionadas para predecir las **bounding boxes** y las **clases** de los objetos, todo en base al prompt original.



Innovaciones de Grounding Dino

Detección sin anchors

A diferencia de métodos clásicos, no requiere cajas predefinidas, lo que simplifica y flexibiliza el proceso.

Sin NMS necesario

Evita el post-procesamiento de Non-Maximum Suppression, optimizando la predicción de cajas.

Queries guiadas por lenguaje

El texto dirige directamente qué y dónde buscar en la imagen.

Atención cruzada

Permite una profunda interacción entre las representaciones visuales y lingüísticas.

Vocabulario abierto

COCO: 80 clases fijas → **Grounding DINO:** ∞ clases
(cualquier texto describible)

Casos de uso y limitaciones de Grounding DINO

Casos de uso:

- **Inspección industrial sencilla:** "defectos en soldadura", "tornillos faltantes", "grietas en superficie"
- **Retail y e-commerce:** "productos dañados", "etiquetas de precio", "artículos mal colocados"
- **Seguridad:** "personas sin casco", "vehículos en zona restringida", "objetos abandonados"
- **Agricultura:** "frutos maduros", "plagas en hojas", "equipos agrícolas"

Limitaciones

- **Ambigüedad lingüística:** Prompts vagos ("cosa roja") pueden generar detecciones imprecisas
- **Objetos muy pequeños:** Rendimiento reducido en objetos menores al 1% del área de imagen
- **Escenas complejas:** Dificultades con múltiples objetos superpuestos o muy similares
- **Dependencia del lenguaje:** Requiere descripciones precisas; no funciona bien con jerga técnica muy específica

Anatomía de un VLM: ¿Cómo ve el LLM?



Vision encoder (Ojos)

Extrae **features visuales** de la imagen (ej. ViT o SigLIP), transformando píxeles en una representación numérica de alto nivel.



Projector/Adapter (Traductor)

Una capa (MLP o Linear) que convierte las **features visuales** en 'Tokens Visuales', adaptándolos para que tengan el mismo tamaño y formato que los tokens de texto.



LLM Backbone (Cerebro)

Recibe [Tokens de Sistema] + [Tokens Visuales] + [Tokens de Usuario] como una única secuencia. La imagen se convierte en una secuencia de tokens pseudo-palabras que se concatenan al prompt de texto.

VLM (Qwen2.5/3-VL): conversa y razona sobre imágenes

Los **Vision-Language Models** (VLM) como Qwen2.5-VL llevan la multimodalidad al extremo: además de describir o detectar, también **razonan, comparan, extraen datos estructurados** y mantienen conversaciones sobre contenido visual.

Capacidades avanzadas:

- **OCR con comprensión:** lee texto y entiende su significado y estructura (facturas, formularios, diagramas)
- **Análisis de gráficos:** interpreta charts, tablas y visualizaciones, extrayendo insights
- **Instrucciones paso a paso:** "explica cómo funciona este mecanismo" o "identifica errores en este montaje"
- **Salida estructurada:** puede devolver JSON con campos específicos que solicites
- **Resolución Dinámica (Naive Dynamic Resolution):** A diferencia de modelos anteriores, procesa imágenes con cualquier relación de aspecto y resolución nativa, clave para leer documentos y textos pequeños sin deformarlos

Trade-off importante: Los VLMs son mucho más pesados y lentos que CLIP o BLIP. Úsalos **solo donde aporten valor real**, no como solución universal. Para tareas simples, CLIP o Grounding DINO son suficientes.

Mejores prácticas de prompting

Para maximizar la eficacia de los modelos multimodales, hay que formular los prompts de manera estratégica y específica para cada tipo de modelo.

Para CLIP

- Usar templates como "una foto de {}"
- Múltiples formulaciones del mismo concepto
- Evitar negaciones directas en el prompt

Para Grounding DINO

- Separar conceptos con " . " (punto)
- Ser descriptivo pero conciso
- Incluir atributos visuales clave

Para Qwen/VLMs

- Instrucciones claras y específicas
- Especificar formato de salida (ej. JSON)
- Dar ejemplos concretos si es posible

✗ Malos prompts

- "¿Está bien?"
- "no hay personas sin casco"

✓ Buenos prompts

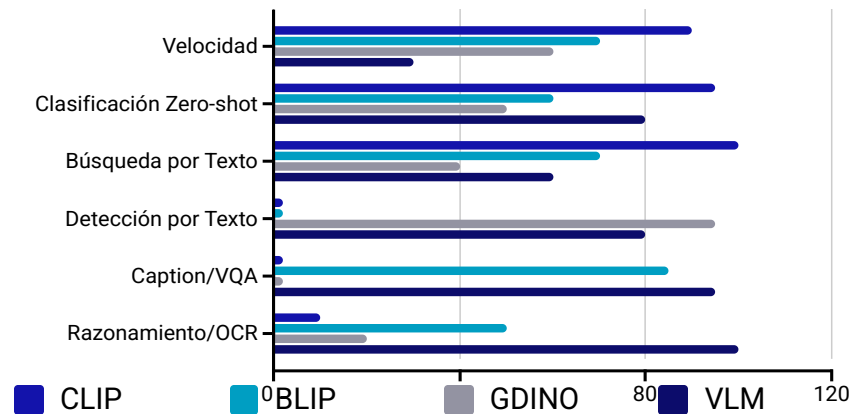
- "Lista todos los defectos visibles en esta pieza metálica"
- "personas con casco . personas sin casco" (para comparar)

¿Qué modelo uso para cada tarea?

La elección del modelo correcto depende de la **complejidad de la tarea** y las **restricciones de latencia**:

Regla de decisión rápida:

- ¿**Velocidad y filtrar**? → CLIP
- ¿**Descripciones o Q&A simple**? → BLIP
- ¿**Localizar objetos por texto**? → Grounding DINO
- ¿**Razonamiento complejo o datos estructurados**? → VLM



Limitaciones y buenas prácticas

Los modelos multimodales son muy potentes pero tienen **limitaciones importantes** que debes tener en cuenta:

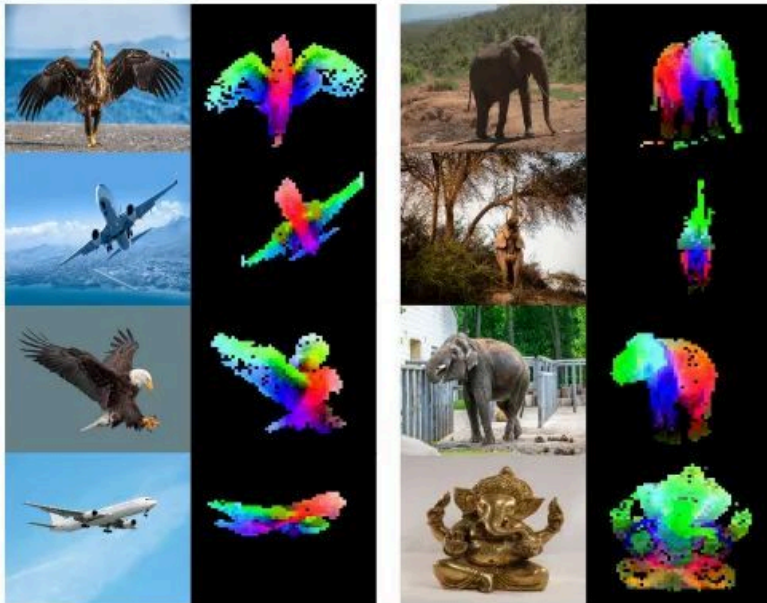
📄 ⚠️ **Alucinaciones:** Los modelos generativos pueden inventar detalles que no existen en la imagen. Especialmente común en VLMs cuando preguntas sobre aspectos ambiguos.

📄 ⚠️ **Sesgos:** Los datos de entrenamiento contienen sesgos sociales y culturales que se reflejan en las predicciones. Valida siempre con datos representativos de tu población objetivo.

📄 ⚠️ **Prompts ambiguos:** "Encuentra el problema" puede dar resultados inconsistentes.

Estrategias de mitigación:

- **Ajusta thresholds:** Empieza con confianza 0.5, sube si hay muchos FP, baja si hay muchos FN
- **Valida con tus datos:** Los benchmarks públicos no garantizan rendimiento en tu dominio
- **Human-in-the-loop:** En casos críticos (medicina, seguridad), siempre revisa las predicciones
- **Prompting específico:** Evita negaciones ("sin defectos"), prefiere afirmaciones ("tornillos en buen estado"). Pide formato de salida explícito (JSON con campos concretos)



4. DINO (v1, v2, v3): ver sin etiquetas

Representaciones visuales generales para muchas tareas

DINO (Self-Distillation with NO labels) y sus evoluciones representan un **paradigma radicalmente diferente**: aprender representaciones visuales potentes *directamente de los píxeles*, sin necesidad de etiquetas humanas.

¿Por qué auto-supervisado?

El aprendizaje auto-supervisado ofrece una vía para construir modelos de visión robustos y escalables, superando las barreras del etiquetado manual.

Aprendizaje supervisado

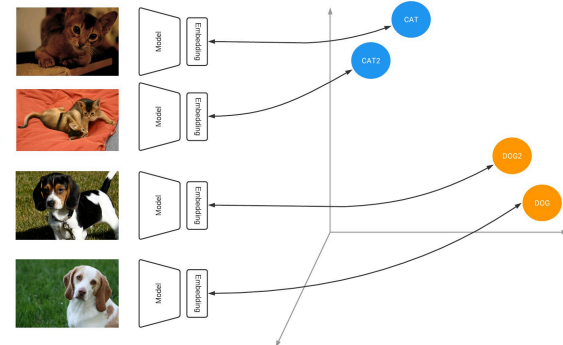
- Necesita **etiquetas manuales** de expertos.
- **Costoso y lento** de producir.
- Limitado a **categorías predefinidas**.
- No escala bien a grandes volúmenes de datos.

Ejemplo: ImageNet

- 14 millones de imágenes.
- 1000 categorías.
- Años de trabajo manual.
- Millones de dólares de inversión.

Aprendizaje auto-supervisado

- **No requiere etiquetas** manuales.
- Aprende de la **estructura interna** de los datos.
- Escala a **billones de imágenes** de manera eficiente.
- Genera **representaciones más generales** y transferibles.



Self-supervised learning explicado

El modelo genera sus propias señales de supervisión desde los datos

- **CONTRASTIVE LEARNING**

Imágenes similares → embeddings cercanos

Imágenes diferentes → embeddings lejanos

- **ROTATION PREDICTION**

Rotar imagen

Predecir ángulo de rotación

- **MASKED PREDICTION**

Ocultar partes de imagen

Predecir partes ocultas

- **CLUSTERING**

Agrupar features similares sin saber qué representan

epoch: 0



Auto-Destilación

DINO entrena una red enseñándole a predecir la salida de su versión más estable, la Red Maestra (Teacher), sobre diferentes vistas de la misma imagen. Este proceso fomenta la consistencia y la robustez de las representaciones.



Red Estudiante (Student Network)

- Aprende activamente y ajusta sus pesos mediante backpropagation.
- Recibe diferentes aumentaciones (vistas locales/detalladas) de la imagen.



Red Maestra (Teacher Network)

- Posee la misma arquitectura que la estudiante, pero con pesos estables.
- Sus pesos se actualizan lentamente a través de un promedio móvil (Momentum Update) de los pesos de la red estudiante.
- Recibe una vista global o contextual de la imagen.

El objetivo es que la red estudiante sea capaz de replicar la predicción de la red maestra, forzándola a aprender representaciones visuales de alta calidad sin etiquetas explícitas. Este mecanismo simula una auto-supervisión continua.

Lo que DINO aprende solo

Al entrenar Vision Transformers (ViT) con el algoritmo DINO, el modelo aprende automáticamente a separar el objeto principal del fondo, logrando una representación visual interpretable y universal.



El Descubrimiento: Segmentación Auto-aprendida

DINO permite que el modelo aprenda a segmentar objetos de manera inherente, sin necesidad de anotaciones humanas explícitas o funciones de pérdida de segmentación a nivel de píxel. Es un aprendizaje "viendo" el mundo de forma consistente.



El Poder de la Auto-Atención (Self-Attention)

- **Visión Global:** Cada parte de la imagen construye su comprensión procesando y relacionándose con otras ubicaciones, incluso las más distantes.
- **Comprensión de Escena:** De esta manera, el modelo desarrolla una interpretación rica y de alto nivel de la escena completa.
- **Segmentación por Atención:** Al analizar los mapas de atención internos del modelo, se observa que estos se alinean directamente con regiones semánticas coherentes, es decir, el modelo "separa" los objetos automáticamente.



DINO: Identificación de copias de imágenes

Un hallazgo sorprendente es que DINO es excepcionalmente bueno para identificar copias de imágenes, incluso sin haber sido diseñado para ello. Este modelo podría convertirse en un estándar para sistemas de detección de copias, combatiendo la desinformación y protegiendo los derechos de autor.

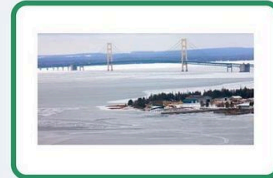
Query



DINO

96.4%

AVERAGE PRECISION



Multigrain architecture

90.7%

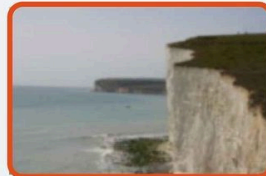
AVERAGE PRECISION



Supervised ViT

89%

AVERAGE PRECISION



Evolución de la familia DINO

DINO, acrónimo de "Self-**D**istillation with **NO** labels", ha marcado un antes y un después en el aprendizaje auto-supervisado en visión artificial.

1

2021: DINO v1

Primer ViT auto-supervisado. Demostró que la atención del modelo podía realizar segmentación de objetos sin supervisión explícita.

2

2022: DINO v2

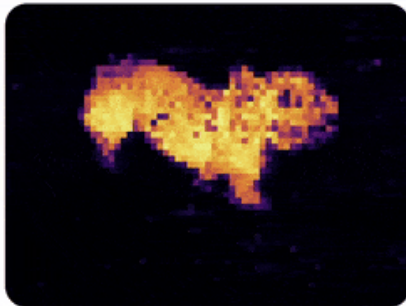
Escalado a **142M imágenes**, con mejoras arquitectónicas. Generó features visuales mucho más robustas y generalizables.

3

2024: DINO v3

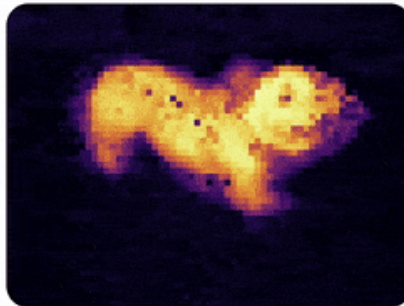
Entrenado con **1B+ imágenes**, logrando una calidad de representación cercana a la supervisada. Base para nuevos modelos fundacionales.

DINO es la base de modelos como SAM y Grounding DINO, proveyendo **features universales** para múltiples tareas de visión.



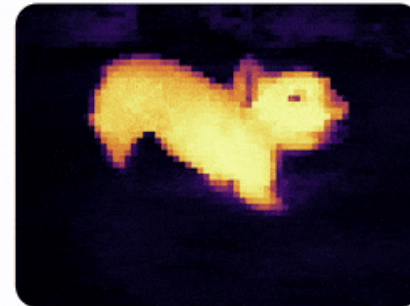
DINO

Initial research proof-of-concept, with 80M-parameter models trained on 1M images.



DINOv2

First successful scaling of a SSL algorithm. 1B-parameter models trained on 142M images.



DINOv3

An order of magnitude larger training compared to v2, with particular focus on dense features.

Pipeline de procesamiento de datos DINOv2

DINOv2 se entrena con un vasto y diversificado conjunto de datos para lograr una representación visual robusta. El pipeline de procesamiento es la clave:

→ Ingesta de datos

Imágenes **curadas** y **no curadas** (de la web) se combinan para diversidad.

→ Generación de embeddings

Todas las imágenes se transforman en **embeddings** de alta dimensión.

→ Eliminar duplicados

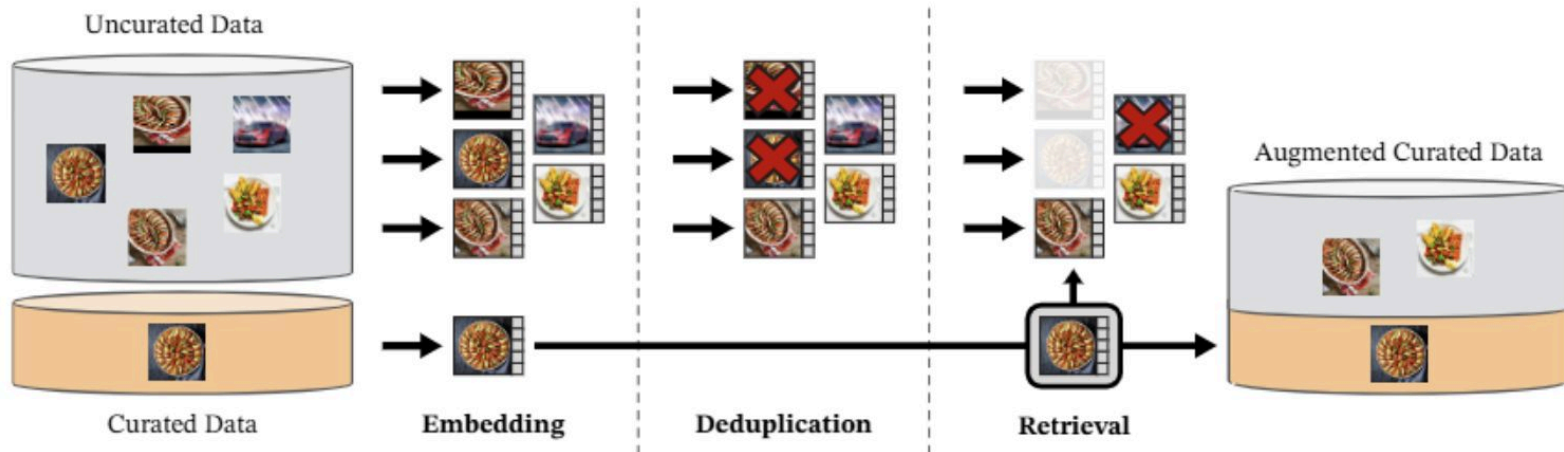
Se eliminan duplicados o imágenes muy similares de los datos no curados usando embeddings.

→ Matching y enriquecimiento

Imágenes no curadas visualmente similares a las curadas se añaden para enriquecer el dataset.

→ Retrieval auto-supervisado

El sistema expande el dataset añadiendo imágenes relevantes sin etiquetado manual.



Cómo funciona DINO

DINO opera mediante un innovador mecanismo de auto-destilación donde una red "estudiante" aprende de una red "maestra" en un ciclo iterativo y sin necesidad de etiquetas. El proceso se desarrolla en los siguientes pasos clave:



1. Imagen y aumentaciones diferentes

Una imagen de entrada se somete a diversas transformaciones (aumentaciones) para crear múltiples vistas, incluyendo "crops" locales y globales.



2. Red estudiante: vistas locales

La red estudiante recibe y procesa específicamente los "crops" locales, o las versiones más pequeñas y detalladas, de la imagen aumentada.



3. Red maestra: vistas globales

Simultáneamente, la red maestra procesa las vistas globales y más amplias de la misma imagen, capturando el contexto general.



4. Estudiante predice salida de la red maestra

El objetivo de la red estudiante es aprender a predecir la representación (salida) de la red maestra a partir de sus propias vistas locales.



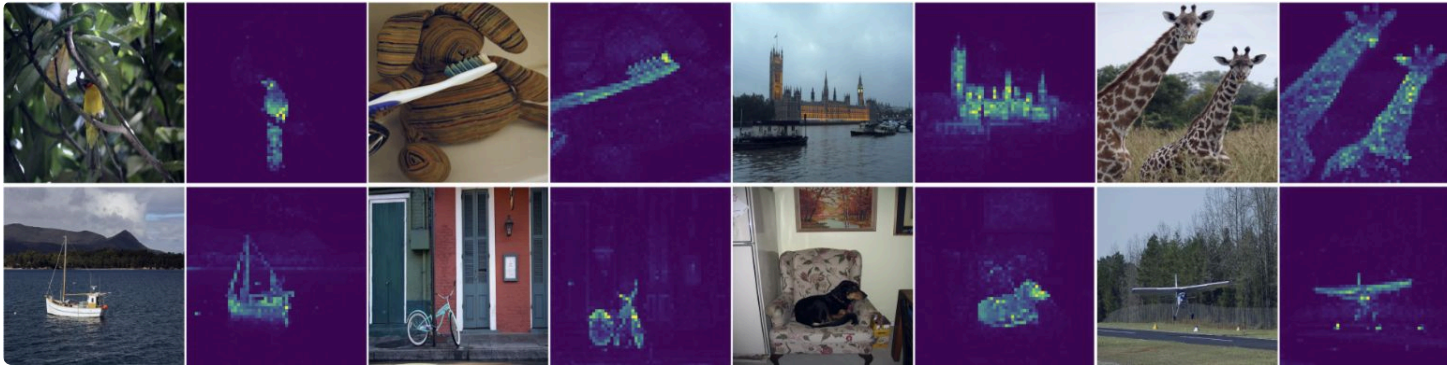
5. Maestra se actualiza por promedio

Los pesos de la red maestra se actualizan de forma suave, utilizando un promedio móvil exponencial (momentum update) de los pesos de la red estudiante, asegurando estabilidad y progreso constante.

Lo que DINO aprende solo

Descubrimientos importantes

DINO es una forma eficiente de aprender representaciones y también revela una serie de capacidades **emergentes*** que no fueron explícitamente entrenadas, demostrando cómo el modelo adquiere una comprensión profunda de la escena visual.



Segmentación semántica

- Los mapas de atención de DINO actúan como **máscaras de objetos**, distinguiendo el primer plano del fondo.
- Identifica objetos coherentes en la escena sin ninguna anotación durante su entrenamiento.

Correspondencias

- Es capaz de encontrar puntos equivalentes y coherentes entre diferentes vistas de una imagen o incluso entre imágenes distintas.
- Esta capacidad surge sin haber sido supervisado nunca para tareas de correspondencia.

Detección de objetos

- Los *heads* de atención de DINO se focalizan naturalmente en objetos salientes dentro de la imagen.
- Esto ocurre sin que el modelo haya visto una sola *bounding box* durante su fase de pre-entrenamiento.

Clasificación

- Utilizando las *features* extraídas por DINO, un clasificador k-NN simple puede lograr buenos resultados.
- DINO no utiliza etiquetas de clases ni para el entrenamiento, pero sus representaciones son altamente discriminativas.

DINO como extractor de características

DINO es una herramienta muy potente para la extracción de características visuales de alta calidad, esenciales para diversas aplicaciones de visión artificial, aprendiendo sin etiquetas.

Uso principal

"Backbone" para extraer características visuales universales de cualquier imagen, sin anotaciones.

Ventajas

- Generales y transferibles.
- Alto rendimiento en dominios nuevos.
- No sesgado a categorías específicas.

Calidad

- **Densas** (por "patch").
- **Semánticamente significativas**.
- Altamente discriminativas.

Aplicaciones

- Clasificación (k-NN).
- Detección de anomalías.
- Clustering visual.



Entendiendo qué aprende DINO

Para comprender cómo DINO procesa y entiende el mundo visual, utilizamos diversas técnicas de visualización que revelan la riqueza de sus representaciones auto-supervisadas.

Mapas de atención

Visualizan las ponderaciones de auto-atención del modelo, mostrando explícitamente dónde se focaliza la red. Revelan segmentación implícita de objetos y la relación entre elementos.

PCA de features

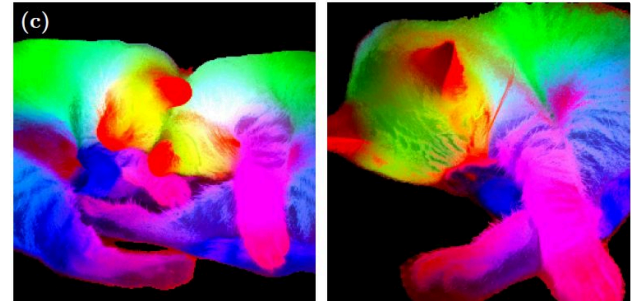
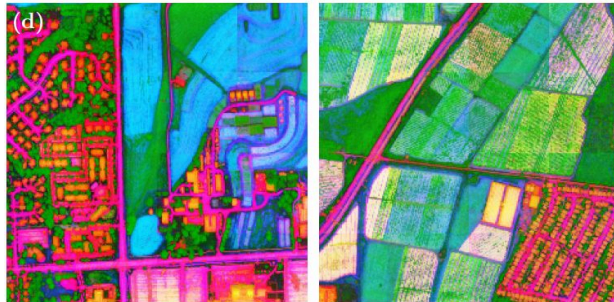
Aplica Análisis de Componentes Principales a las características densas extraídas por DINO. Se mapean los primeros 3 componentes a canales RGB para visualizar agrupaciones semánticas.

t-SNE / UMAP

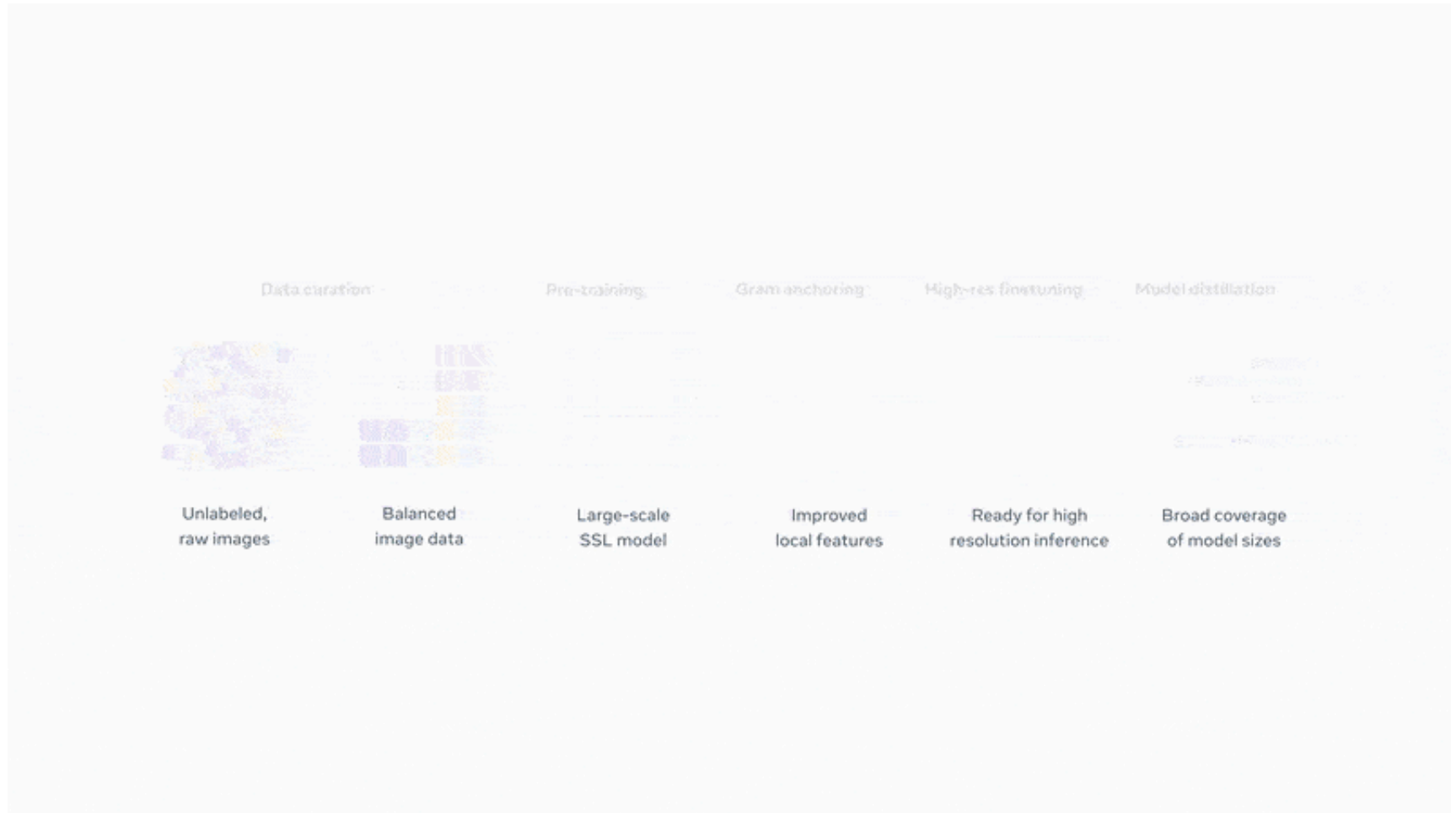
Estas técnicas de reducción de dimensionalidad no lineal proyectan los embeddings de alta dimensión en un espacio 2D. Muestran cómo DINO agrupa conceptos visuales similares sin etiquetas.

Vecinos cercanos

Identifica parches de imagen o imágenes completas con las representaciones DINO más similares. Útil para validar la coherencia de las características y encontrar correspondencias visuales.

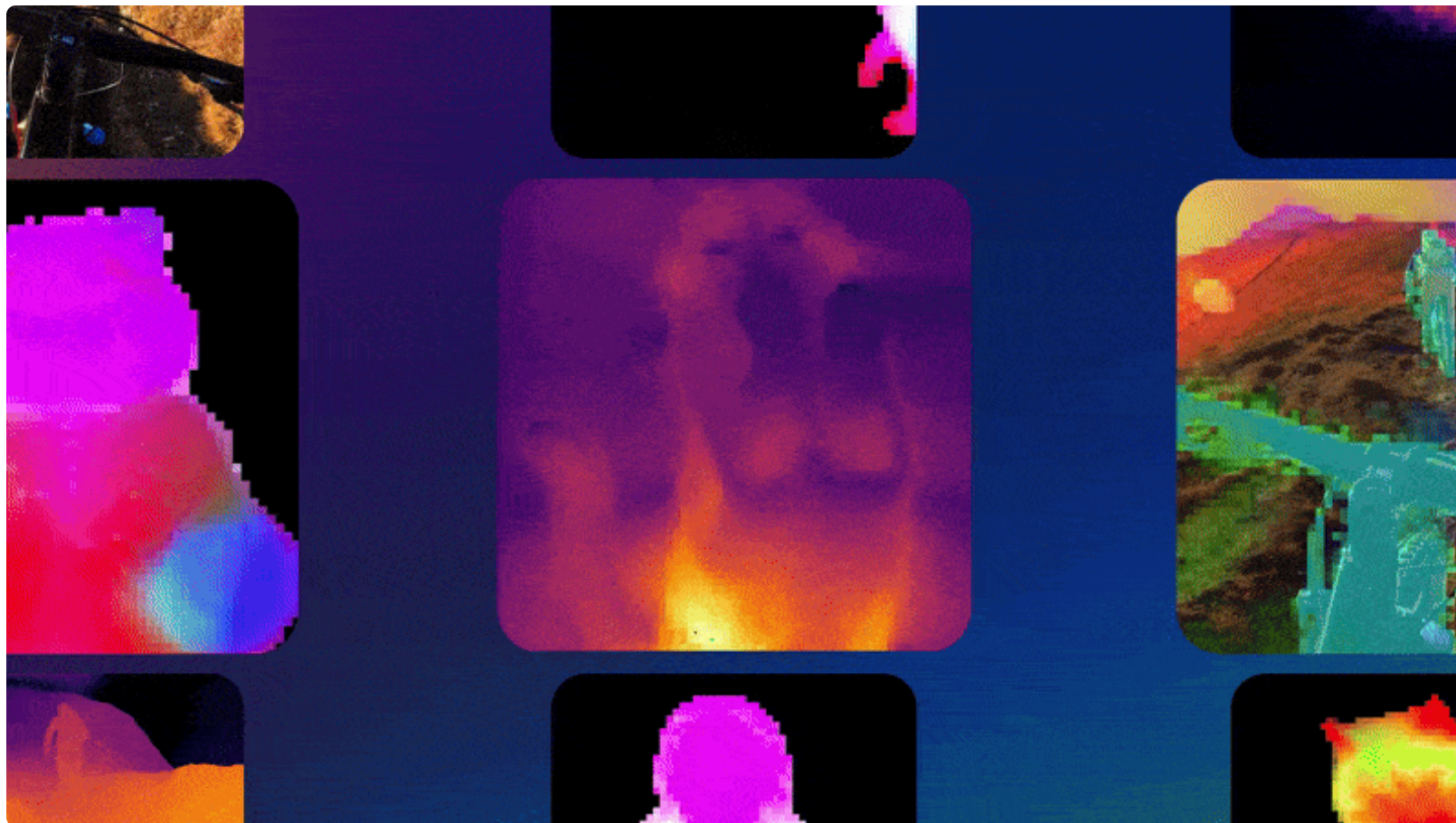


Dino v3



*GIF Animado

Dino v3



Cuándo usar (y cuándo no) DINO

DINO no es una solución universal. Úsalo cuando tenga sentido estratégico:

✓ Úsalo si...

- **No tienes etiquetas** o son muy costosas de obtener
- Las clases/categorías **cambian frecuentemente** y re-entrenar es inviable
- Necesitas **features reutilizables** para múltiples tareas (segmentación, anomalías, búsqueda...)
- Quieres **descubrir patrones** sin asumir taxonomía previa
- Buscas **robustez** a cambios de dominio (iluminación, fondo, escala)

✗ Evítalo si...

- Tienes **muchas etiquetas fiables** y un objetivo de clasificación bien definido (supervisado será mejor)
- Requieres **tiempo real en edge** con hardware muy limitado (DINO es más pesado que MobileNet)
- Necesitas **explicabilidad detallada** y reglas interpretables (modelos más simples o clásicos)
- El problema es extremadamente específico y las features generales no aportan (raro, pero posible)

Regla empírica: Si tienes etiquetas abundantes y un objetivo fijo, entrena supervisado. Si no, empieza con DINO features y ahorra semanas de trabajo.

Anomalías con DINOv2/v3 (estilo PatchCore)

La detección de anomalías de vital importancia en contextos donde los "fallos" son raros o impredecibles. DINO, combinado con un enfoque tipo PatchCore, ofrece un método robusto y eficiente para identificar desviaciones sin necesidad de etiquetas de anomalía.



Imágenes de referencia 'normales'

Se recopilan entre 10 y 50 imágenes que representan el estado "normal" o "sin defectos" del objeto o proceso a monitorear.



Extracción de features DINO

Se utiliza DINOv2 (ViT-B/14 para un buen balance) o DINOv3 (para mayor robustez) como extractor de características.



Embeddings por parche

Se extraen los tokens de atención (no solo el CLS) y se normalizan con L2, capturando representaciones detalladas para cada "parche" de la imagen.



Generación del banco de normalidad (CoreSet)

A partir de los embeddings de las imágenes normales, se construye un CoreSet (un subconjunto representativo del 1-10%) para almacenar la "normalidad".



Imagen de test

Una nueva imagen, que podría contener anomalías, se procesa de la misma manera para extraer sus embeddings por parche.



Predicción de anomalías (kNN)

Se calcula la distancia kNN (L2 o coseno) de cada parche de la imagen de test a su vecino más cercano en el CoreSet. Esto genera un heatmap de anomalías y un score global.



5. SAM: Segment Anything (v1, v2, v3)

Segmentación universal en imagen y vídeo, con prompts

SAM (Segment Anything Model) es la evolución del modelo de segmentación más versátil: segmenta **cualquier objeto en cualquier imagen** mediante prompts simples (puntos, cajas, máscaras), y mantiene **coherencia temporal en vídeo** sin entrenar para clases específicas.

¿Qué es segmentación?

La segmentación de imágenes es una tarea básica en visión artificial que va más allá de la simple detección. Su objetivo es asignar una etiqueta de clase a cada píxel de una imagen, delimitando el contorno exacto de los objetos y el fondo.

Tipos de segmentación

- **Semántica**

- Cada píxel se clasifica en una categoría (ej., "coche", "persona", "fondo").
- No distingue entre instancias individuales del mismo tipo de objeto (todos los coches reciben la misma máscara y etiqueta).

- **Por instancias**

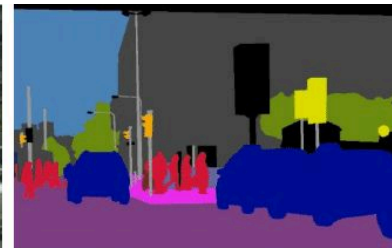
- Cada objeto individual detectado recibe una máscara única y su propia etiqueta.
- Permite distinguir entre objetos del mismo tipo, como "coche 1", "coche 2", "persona A", "persona B".

- **Panóptica**

- Combina la segmentación semántica y por instancias.
- Asigna una máscara única y una etiqueta a cada instancia de objeto cubriendo toda la imagen. Es la más completa pero también la más compleja.



(a) Image



(b) Semantic Segmentation



(c) Instance Segmentation



(d) Panoptic Segmentation

Arquitectura de SAM 1

Un diseño eficiente basado en tres componentes desacoplados

Image encoder (el componente pesado)

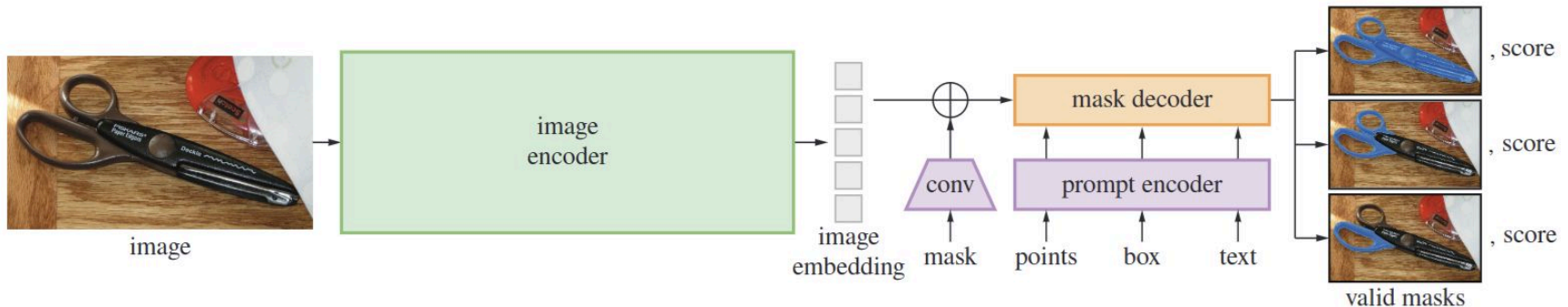
- Basado en Vision Transformer (ViT-H) pre-entrenado con MAE.
- Se ejecuta una única vez por imagen, generando un embedding reutilizable.
- Costoso computacionalmente, pero desacoplado de la interacción.

Prompt encoder (el componente ligero)

- Codifica entradas en tiempo real: puntos, cajas y texto (con positional encodings).
- También procesa máscaras previas (con convoluciones).
- Convierte la intención del usuario en embeddings compatibles.

Mask decoder (el cerebro rápido)

- Arquitectura Transformer ligera (Two-way transformer).
- Realiza Cross-Attention entre embeddings de imagen y prompts.
- Predice 3 máscaras de salida (whole, part, subpart) para resolver ambigüedad.
- Latencia muy baja (~50ms en navegador web).



Arquitectura de SAM 2

Unificando imágenes y vídeo con memoria temporal

Encoder jerárquico (hiera) - La evolución

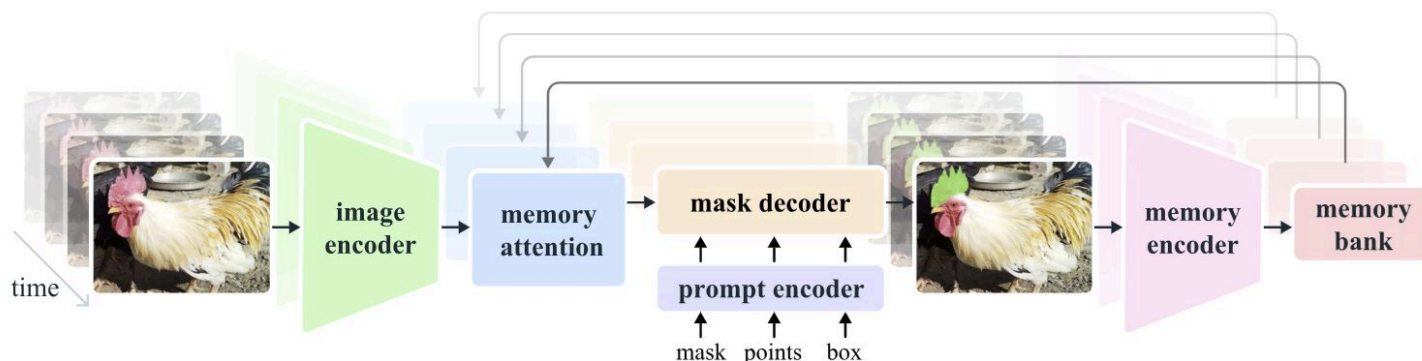
- Sustituye al ViT plano de SAM 1 por una arquitectura jerárquica (Hiera), un enfoque más potente.
- Procesa imágenes en múltiples escalas, capturando detalles finos y contexto global de forma simultánea.
- Mucho más eficiente, permitiendo el procesamiento de vídeo en tiempo real ("Streaming processing").
- Calcula los "features" del fotograma una sola vez, optimizando el rendimiento.

Mecanismo de memoria - La revolución

- Componente clave que permite entender el vídeo y mantener la coherencia temporal.
- **Memory bank:** Almacena información de fotogramas pasados y del fotograma inicial (el "prompt").
- **Memory attention:** Permite al modelo "mirar atrás" para mantener la identidad del objeto a lo largo del tiempo.
- Maneja oclusiones, recordando el objeto incluso si desaparece momentáneamente de la vista.

Decodificador de máscaras & prompting - La interacción

- Recibe tanto los "prompts" del usuario (clics, cajas) como el contexto proporcionado por la memoria.
- Predice máscaras para el fotograma actual y actualiza la memoria para futuros procesamientos.
- **Ambiguity-aware:** Genera 3 opciones de máscara si hay ambigüedad en la segmentación.
- Capaz de corregir la segmentación de todo el vídeo con una única corrección en un solo fotograma (propagación).



Arquitectura de SAM 3

Unificación de detección y tracking con desacoplamiento de reconocimiento

Backbone unificado & prompting multimodal

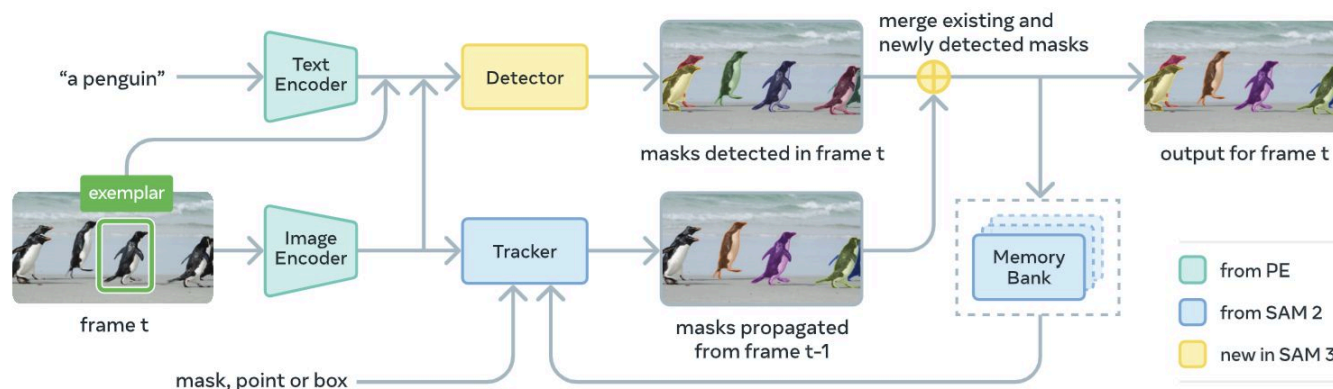
- Utiliza un "Perception Encoder" (PE) compartido, pre-entrenado con alineación visión-lenguaje (contrastive learning).
- **Nuevos prompts:** Acepta texto (frases nominales como "un gato rayado") y "Image Exemplars" (ejemplos visuales positivos/negativos).
- **Fusion encoder:** Un nuevo módulo que condiciona los embeddings de imagen con los tokens del prompt (texto/exemplar) antes de la decodificación.

Diseño dual: detector + tracker

- **Detector (DETR-based):** Procesa cada frame independientemente para encontrar *nuevos* objetos que coincidan con el concepto (texto).
- **Tracker (Herencia SAM 2):** Utiliza el mecanismo de Memoria (Memory Bank/Attention) para propagar identidades temporalmente.
- **Sinergia:** El detector alimenta al tracker con nuevos objetos; el tracker mantiene la coherencia temporal. Si el tracker falla, el detector puede "re-iniciarlo".

Innovación clave: el "Presence Token"

- Soluciona el conflicto entre reconocer ("¿qué es?") y localizar ("¿dónde está?").
- Introduce un token global que predice la probabilidad de que el concepto exista en la imagen ($P(\text{presence})$).
- Las queries de objetos solo se enfocan en la localización condicional ($P(\text{match}|\text{presence})$).
- **Resultado:** Reduce drásticamente los falsos positivos en vocabulario abierto al distinguir entre la existencia y la posición del objeto.

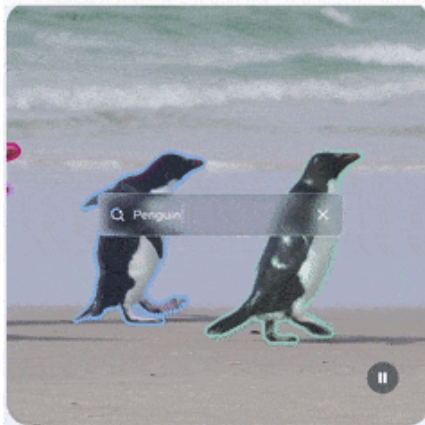


Evolution — SAM → SAM2 → SAM3

La familia de modelos Segment Anything (SAM) ha evolucionado rápidamente, ampliando sus capacidades de segmentación universal desde imágenes estáticas hasta el análisis complejo de vídeo con prompts contextuales.

2025 — SAM3

Modelo unificado que detecta, segmenta y sigue todas las instancias de un concepto (texto o ejemplos) en imagen y vídeo.



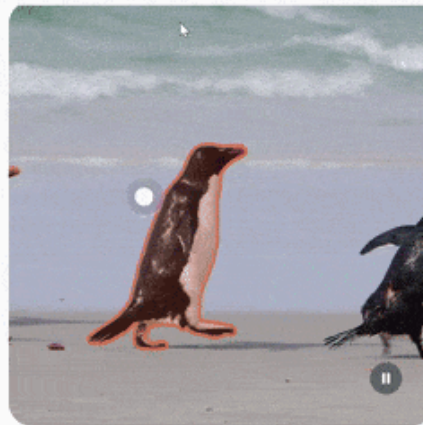
SAM 3

Detect, segment and track every example of any object category in an image or video, using text or examples

2024 — SAM2

Extiende a vídeo con memoria en streaming; más preciso y hasta 6× más rápido que SAM v1 en imagen.

Dataset SA-V: ~51 K vídeos y ~643 K masklets.



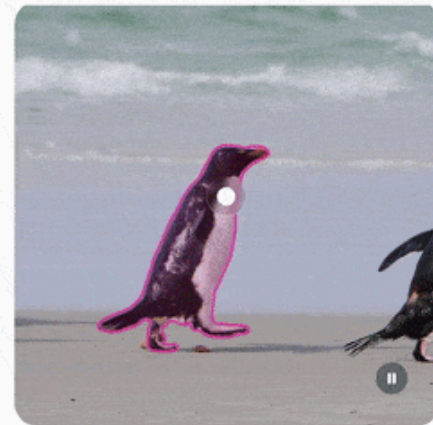
SAM 2

Segment and track any object in any image or video using click, box or mask prompts

2023 — SAM1

Segmentación universal en imagen con prompts (puntos/cajas).

Dataset SA-1B: ~11 M imágenes y ~1.1 B máscaras.



SAM 1

Segment any object in any image with as little as a single click

The data engine: cómo escalar a 1B de máscaras

La creación del dataset SA-1B, clave para el entrenamiento de SAM, se logró mediante un innovador proceso iterativo de "motor de datos", combinando la inteligencia humana con la capacidad de autoaprendizaje del modelo para escalar la anotación.

1. Anotación manual

Expertos humanos realizan anotaciones detalladas en un subconjunto inicial de imágenes. En esta fase, el modelo aún no tiene suficiente conocimiento.



2. Anotación semi-automática

El modelo, pre-entrenado con datos manuales, propone máscaras que los humanos revisan y corrigen. Esta interacción acelera el aprendizaje del modelo, que mejora exponencialmente.

3. Anotación totalmente automática

Una vez que el modelo es robusto, anota imágenes completas de forma autónoma. Solo se aceptan las máscaras con alta confianza, permitiendo un escalado masivo de datos.

Conclusión: SA-1B es 99.1% generado automáticamente por el propio SAM, demostrando el poder del bucle de auto-mejora.

"Promptable segmentation": la idea central

La segmentación "promptable" es la capacidad de un modelo de visión artificial para segmentar objetos en imágenes o vídeos basándose en diferentes tipos de "prompts" (indicaciones). Esto permite una interacción flexible y eficiente sin necesidad de reentrenar el modelo.

Tipos de prompts y sus capacidades

Puntos

- **Múltiples puntos:** Incluye o excluye áreas específicas para refinar la máscara deseada.
- **Precisión:** Útil para objetos ambiguos o con bordes complejos.

Cajas (bounding box)

- **Delimitación amplia:** Dibuja una caja alrededor del objeto; SAM la convierte en una máscara precisa.
- **Rapidez:** Ideal para segmentación rápida de objetos bien definidos.

Máscara inicial

- **Refinamiento:** Proporciona una máscara aproximada (rough) y el modelo perfecciona los bordes y rellena huecos.
- **Perfeccionamiento:** Útil para mejorar segmentaciones previas o manuales.

Texto

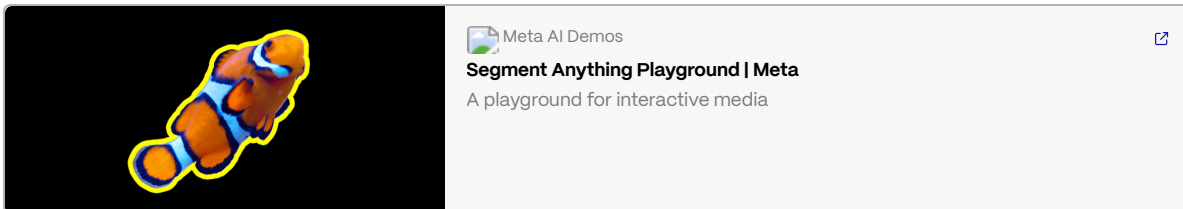
- **SAM1/SAM2:** Mediante adaptadores (CLIP/Grounding DINO), se puede usar texto como "el coche rojo".
- **SAM3 (Nativo):** Interpreta directamente prompts de texto ("el árbol de la izquierda") para generar máscaras.

Ejemplar (Crop)

- **SAM3:** A partir de un recorte de ejemplo, segmenta todas las instancias del mismo concepto en la imagen o vídeo.
- **Generalización:** Ideal para tareas de conteo o seguimiento de objetos idénticos.

Segment everything

- **Cobertura completa:** Genera una máscara para cada objeto detectable en la imagen sin prompts específicos.
- **Análisis exhaustivo:** Útil para la inspección completa de escenas complejas.



Cuándo usar (y cuándo no) SAM

✅ Úsalo si... (ideal para flexibilidad y Zero-Shot)

- **Segmentación "Zero-Shot" crítica:** Necesitas segmentar objetos desconocidos, raros o que no existían en tu dataset de entrenamiento (ej. microscopía, underwater).
- **Video y coherencia temporal:** Requieres rastrear objetos que se mueven, ocultan y reaparecen sin entrenar un tracker específico (Gracias a la memoria de SAM 2/3).
- **Interacción humana (human-in-the-loop):** Estás construyendo herramientas de anotación de datos o edición donde el usuario refina el resultado con clics o cajas.
- **Búsqueda por conceptos (SAM 3):** Necesitas encontrar todas las instancias de un concepto abstracto ("tornillos oxidados") usando texto o ejemplos visuales, no solo puntos.
- **Escasez de datos:** No tienes miles de máscaras anotadas para entrenar un modelo supervisado tradicional.

❌ Evítalo si... (mejor usar modelos supervisados clásicos)

- **Restricciones de hardware extremas (Edge):** Necesitas inferencia en milisegundos en dispositivos IoT o móviles antiguos. Aunque eficiente, SAM 3 requiere GPUs potentes (H200) para latencia de ~30ms en cargas altas.
- **Entorno cerrado y repetitivo:** Si tu problema es "detectar botellas en una cinta transportadora" (clases fijas, fondo controlado), un modelo ligero como RF-DETR-Seg será mucho más rápido y barato de desplegar.
- **Razonamiento complejo:** SAM 3 entiende "frases nominales simples". Si necesitas lógica compleja, necesitas un VLM.



Otras tareas fundacionales de visión artificial

Bonus extra

→ Estimación de pose

Identificar y localizar puntos clave del cuerpo humano o de objetos en imágenes y vídeos.

→ OCR moderno

Extraer texto de imágenes y documentos, transformándolo en datos editables y buscables.

→ Super-resolución

Mejorar la calidad y resolución de imágenes o vídeos, a menudo a partir de datos de baja resolución.

→ Background removal

Segmentar y eliminar automáticamente el fondo de una imagen, aislando el objeto principal.

→ Estimación de profundidad

Calcular la distancia de los objetos en una escena respecto a la cámara, creando mapas de profundidad.

→ Image matching

Encontrar correspondencias o similitudes entre dos o más imágenes para tareas como la localización o el reconocimiento.

Estimación de pose humana: de píxeles a esqueletos

Estructurando el cuerpo humano en coordenadas digitales

¿Qué es?

Tarea de visión que localiza articulaciones clave (*keypoints*) para modelar la postura y movimiento.

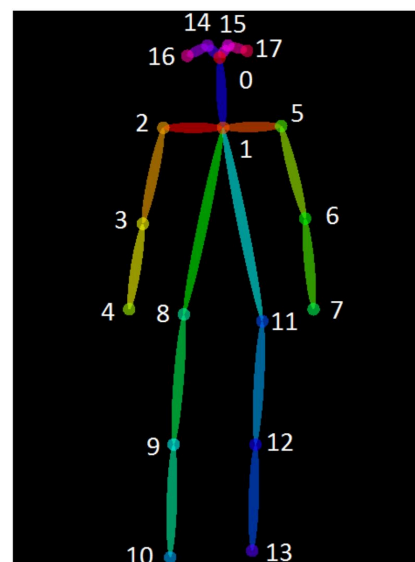
El estándar (COCO-17):

Se buscan típicamente 17 puntos:

- Cabeza: Nariz, Ojos (2), Orejas (2).
- Tren Superior: Hombros (2), Codos (2), Muñecas (2).
- Tren Inferior: Caderas (2), Rodillas (2), Tobillos (2).

El output:

Una lista de coordenadas (x, y) más un *score* de confianza por punto.



- ❏ Transformamos una matriz de píxeles ininteligible en una estructura geométrica analizable (*Skeleton*).

Enfoques de modelado: Top-down vs. Bottom-up

Trade-off entre precisión y velocidad en multitudes

Top-Down (alta precisión)

Proceso: Primero detecta personas (cajas), luego estima la pose dentro de cada una.

Ejemplos: HRNet, VitPose.

- **Pros:** Máxima precisión, robustez a distintas escalas.
- **Contras:** Lento en grandes aglomeraciones (coste lineal con el número de personas).

Bottom-Up (alta velocidad)

Proceso: Detecta todos los puntos clave de la imagen a la vez, luego los agrupa en personas.

Ejemplo: OpenPose.

- **Pros:** Velocidad constante, independiente del número de personas.
- **Contras:** Menos preciso, el agrupamiento de puntos es complejo.

Single-Stage (el equilibrio moderno)

Proceso: Redes como YOLO-Pose predicen cajas y puntos simultáneamente en una sola pasada (End-to-End).

- **Ventaja:** Velocidad en tiempo real con precisión competitiva, ideal para dispositivos "Edge".

Modelos destacados y evaluación

Herramientas prácticas para cada escenario

1

MediaPipe (Google)

Funciona eficientemente en CPU. Ideal para aplicaciones de fitness, filtros AR o escenarios de una sola persona. EL problema es que solo da los keypoints de una sola persona. También sirve para manos y cara.

2

YOLOv8-Pose (ultralytics!!)

El estándar industrial actual. Ofrece un equilibrio óptimo entre velocidad y precisión. Excelente para detección multi-persona y es fácil de entrenar y adaptar.

3

OpenPose (CMU)

El pionero académico que estableció las bases. Aunque influyente, hoy en día es computacionalmente pesado en comparación con opciones más modernas.

4

HRNet

Prioridad en precisión extrema. Cuando la exactitud es crítica (ej. análisis médico, biomecánico) y la velocidad de inferencia es secundaria.

¿Cómo medimos la calidad? La métrica OKS

OKS (Object Keypoint Similarity) es el equivalente a IoU para esqueletos. No mide la distancia en píxeles de forma bruta, sino una "similitud perceptual" entre los puntos clave detectados y los reales.

- ☐ El OKS penaliza de forma inteligente: Un error de 5 píxeles en un punto "rígido" como el ojo penaliza mucho más que un error de 5 píxeles en una articulación "flexible" como la muñeca o la cadera.



Impacto real y desafíos técnicos

Dónde genera valor la estimación de pose



Sport & fitness

- **"Entrenador IA":** Conteo preciso de repeticiones (sentadillas, flexiones...).
- **Análisis de técnica:** Medición de ángulos corporales para prevenir lesiones.
- Aplicaciones en plataformas como Nike Training o Peloton.



Seguridad industrial & ergonomía

- **Detección de posturas peligrosas:** Identificar movimientos de riesgo al levantar peso.
- **"Hombre caído":** Alertas de seguridad en tiempo real en entornos peligrosos.
- **Optimización:** Mejora de la eficiencia en líneas de montaje (ej. Toyota/BMW).



Salud & rehabilitación

- **Fisioterapia remota:** Validación de ejercicios en casa sin supervisión presencial.
- **Análisis de marcha:** Detección temprana de patologías neurológicas o musculoesqueléticas.



El gran reto: oclusiones y reconstrucción 3D

El problema: La estimación de pose se complica cuando partes del cuerpo se ocultan entre sí o por objetos externos, perdiendo información crucial.

La solución: Modelos avanzados emplean "memoria temporal" para rastrear y predecir oclusiones, o realizan inferencia 3D para "imaginar" las partes ocultas basándose en la anatomía y el contexto.

OCR moderno: de leer caracteres a entender documentos

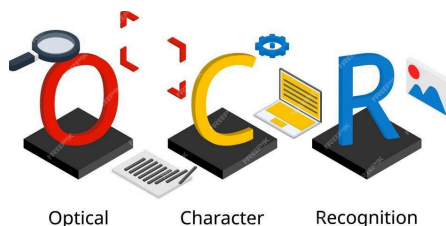
La evolución de Tesseract a los transformers (TrOCR)

El pasado: enfoque clásico (Tesseract)

- **Metodología:** Basado en reglas heurísticas y detección de contornos para identificar caracteres individuales.
- **Limitaciones comunes:**
 - Falla con texto curvo, distorsionado o en perspectivas inusuales.
 - Poca robustez ante fuentes manuscritas complejas o variaciones tipográficas.
 - Sensible al ruido de fondo, baja resolución o iluminación deficiente.
 - Requiere pre-procesamiento intensivo (normalización, binarización).

El presente: transformers para OCR (TrOCR/VLMs)

- **Arquitectura end-to-end:**
 - Utiliza Vision Transformers (ViT) para codificar la imagen de forma global.
 - Un modelo de lenguaje (como BERT o GPT) decodifica la secuencia de texto resultante.
- **Ventaja fundamental:** No "lee" letra a letra, sino que "entiende" la secuencia visual completa del texto en contexto, de forma similar a como los humanos leen.
- **Estado del arte (SOTA):**
 - **TrOCR (Microsoft):** Un referente específico para tareas de OCR.
 - **Qwen 2.5/3-VL:** Un VLM avanzado que integra OCR con capacidades de razonamiento multimodal.



Herramientas prácticas de OCR

Implementación rápida con Hugging Face y EasyOCR



EasyOCR: el comodín versátil

Una opción todo terreno para cualquier proyecto. Es ligera, soporta más de 80 idiomas y funciona eficientemente en CPU. Ideal para prototipos rápidos y escenarios donde no se dispone de GPU.



TrOCR (Hugging Face): precisión avanzada

Ofrece la máxima precisión, especialmente diseñado para textos manuscritos, históricos o muy específicos. Requiere una GPU para un rendimiento óptimo debido a su arquitectura basada en Transformers.



VLMs (Qwen/GPT-4V): documentos estructurados

Perfectos para casos de uso complejos como el procesamiento de facturas o tablas. Permiten extraer información estructurada (e.g., JSON) en lugar de solo texto plano, integrando visión y lenguaje para un razonamiento avanzado.

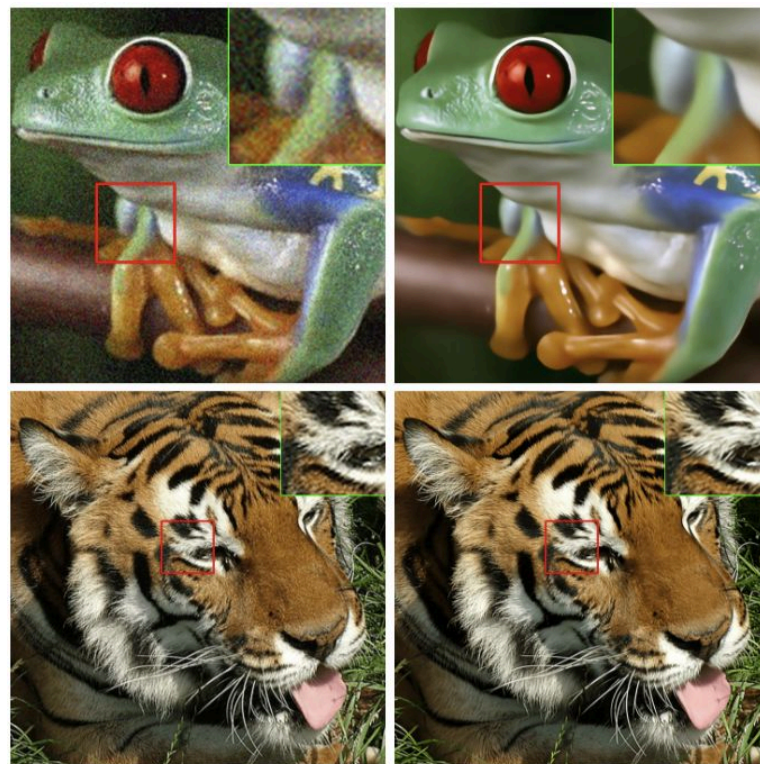
Super-resolución: restaurando el detalle perdido

El uso de Deep Learning para generar detalles plausibles de alta frecuencia a partir de una imagen de baja resolución, superando los límites de la interpolación tradicional (bilineal/bicubica...).

El modelo SOTA: Swin2SR

- Basado en **Swin transformers V2**, lo que le permite procesar imágenes de manera eficiente y global.
- Entrenado para escalar la imagen y también para eliminar artefactos de compresión JPEG y ruido.
- **Ventaja clave:** Recupera y sintetiza texturas finas (como pelo, tela o detalles faciales) que los métodos clásicos dejarían borrosos o artificiales.

❏ **Nota importante:** La super-resolución no es magia. El modelo "imagina" detalles basándose en patrones aprendidos de su vasto conjunto de entrenamiento, no reconstruye información que nunca existió.



LR Bicubic

Swin2SR (ours)

Aplicaciones reales y limitaciones

Dónde aplicar superresolución en industria



Restauración de archivos

- Digitalización de fotos antiguas o documentos históricos dañados.
- Mejora de contenido multimedia antiguo (remasterización).



Medicina e imagen satelital

- Mejorar la legibilidad de *scans* médicos.
- *Zoom* en imágenes aéreas o satelitales.



E-Commerce y marketing

- Mejorar fotos de productos de baja calidad enviadas por usuarios o proveedores.
- Lograr una apariencia profesional en la web para el catálogo de productos.

 **Riesgo crítico:** "alucinaciones". El modelo puede inventar un tumor o una mancha. Nunca usar para diagnóstico crítico sin supervisión.

Eliminación de fondo profesional

El reto del "Alpha Matting" y el modelo RMBG

El problema: bordes difusos

Segmentar objetos sólidos es fácil. Pero, elementos como el pelo, el pelaje o el vidrio transparente son extremadamente difíciles de separar debido a sus **bordes difusos y semitransparentes**.

La solución: **RMBG-1.4 (BRIA AI)**

Un modelo de vanguardia diseñado para el **Alpha Matting**. Genera un canal Alpha de transparencia gradual en los bordes, no una máscara binaria. Esto permite recortes profesionales que se integran perfectamente en cualquier fondo, sin el efecto "pegatina".

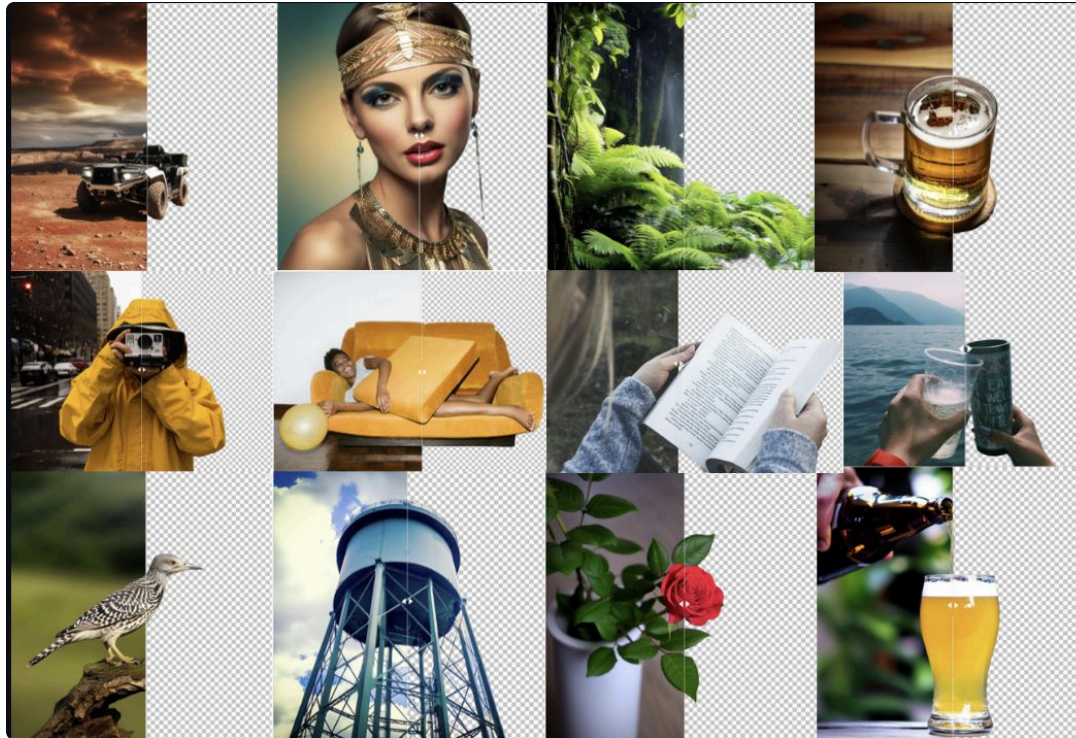


Image matching: encontrando parejas visuales

El fin de los algoritmos heurísticos (SIFT/ORB...)

¿Qué es? Encontrar puntos idénticos en dos fotos de la misma escena tomadas desde ángulos diferentes, a pesar de cambios en la perspectiva o la iluminación. Esta tarea es fundamental para aplicaciones como la reconstrucción 3D, la realidad aumentada y la localización de robots.

La nueva era: deep learning



Detector (SuperPoint/Disk)

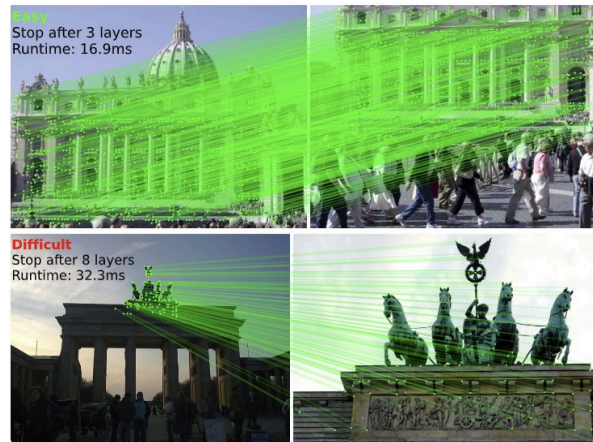
Una red neuronal avanzada que identifica los puntos clave y más distintivos (esquinas, texturas complejas) en una imagen, superando con creces la precisión de los algoritmos matemáticos clásicos.



Matcher (LightGlue)

Un modelo basado en Transformer que evalúa los puntos detectados en ambas imágenes y determina cuáles corresponden entre sí, filtrando eficazmente los errores geométricos y emparejamientos incorrectos.

Ventaja clave: Este enfoque moderno funciona de muy bien incluso con cambios de luz (día/noche), grandes variaciones de perspectiva, oclusiones, ofreciendo una gran robustez.



Reconstruyendo el mundo en 3D

Aplicaciones críticas del matching



Fotogrametría (SfM)

Al identificar y conectar puntos coincidentes en múltiples fotos de una escena, se puede reconstruir la geometría 3D del objeto y la posición de la cámara en cada toma.



SLAM (robótica)

Un robot utiliza el "matching" de características para comprender su posición dentro de un entorno y construir un mapa del mismo, incluso mientras se mueve y explora.



Panoramas (stitching)

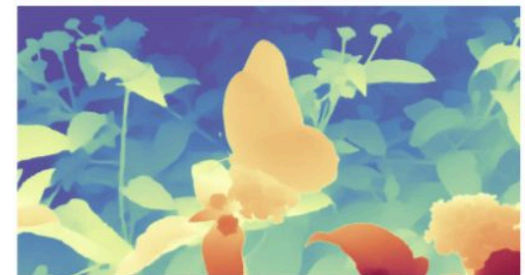
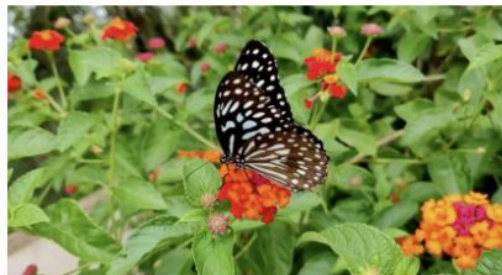
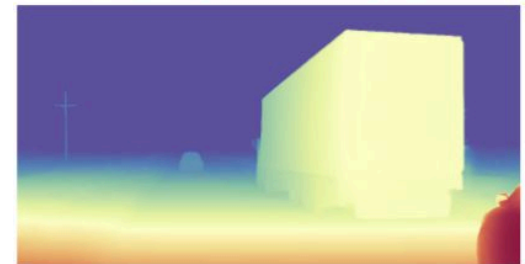
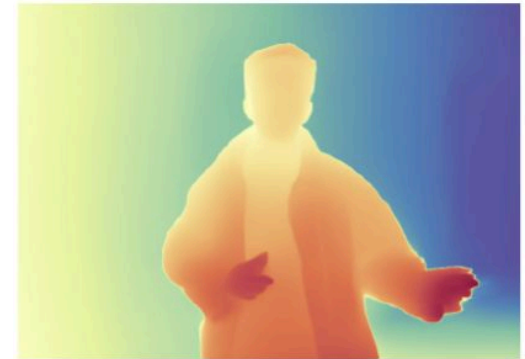
Permite fusionar sin problemas varias imágenes superpuestas en una única foto de gran formato o una vista de 360 grados, eliminando duplicados y alineando las perspectivas.

Depth estimation: 3D desde una sola imagen

La revolución de la profundidad monocular

Depth Anything V2/v3

- Un modelo fundacional entrenado con millones de imágenes etiquetadas y no etiquetadas, aprovechando la diversidad del mundo real.
- **Capacidad:** Genera un "mapa de profundidad" (heatmap) donde el color indica la distancia relativa, revelando la estructura 3D subyacente.
- **Robustez:** Funciona en una gran variedad de escenarios: interiores complejos, exteriores dinámicos, y con elementos desafiantes como objetos transparentes o superficies reflejantes.



Aplicaciones depth

¿Para qué sirve un mapa de profundidad?



Efecto retrato (Bokeh)

Desenfocar el fondo sintéticamente, creando imágenes con una profundidad de campo profesional y destacando el sujeto principal.



Robótica y navegación

Permitir que robots y vehículos autónomos detecten obstáculos y comprendan su entorno 3D, utilizando solo una cámara de bajo coste.



Generación de nubes de puntos

Convertir una foto plana en una estructura 3D navegable (malla o nube de puntos) al combinar la imagen RGB con su mapa de profundidad.



Filtro de imagen

Podemos eliminar partes de la imagen que no están en la escena que nos interesa aplicando un threshold.

📄 **Profundidad relativa vs métrica.** Estos modelos suelen estimar profundidad relativa (mapas de calor). Para obtener metros reales (métrica), se necesita calibración de cámara o modelos específicos (Metric Depth Estimation).



¡Muchas gracias!

Cualquier duda podéis contactarme por email:
s.villanuevalopez@gmail.com